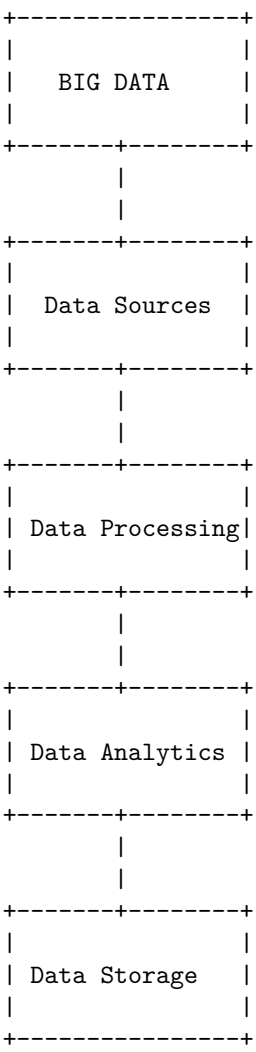
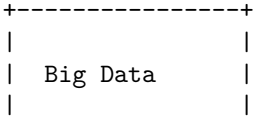
Big Data

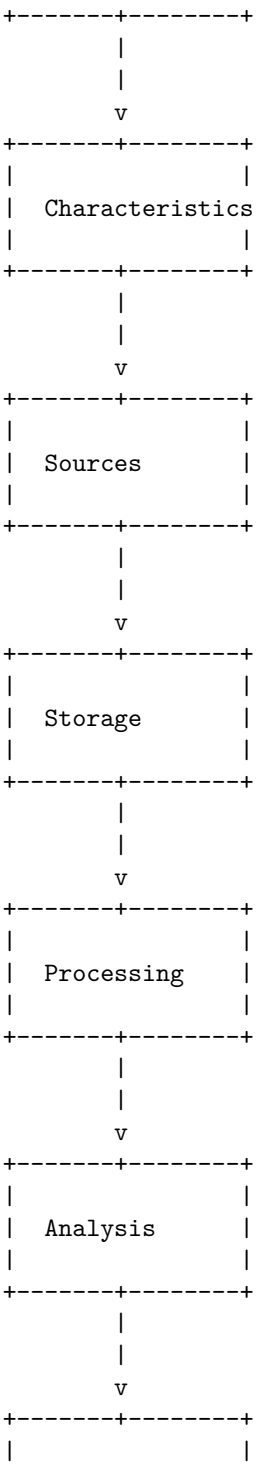
Here is an ASCII diagram that represents the concept of Big Data:



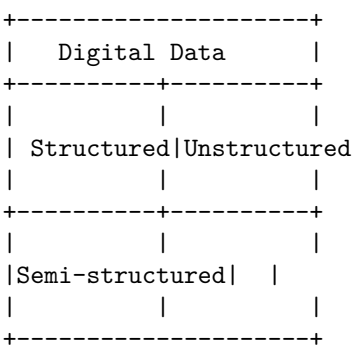
Unit 1 - Introduction to Big Data

Here is an ASCII diagram that provides an overview of the key concepts in Unit 1 - Introduction to Big Data:



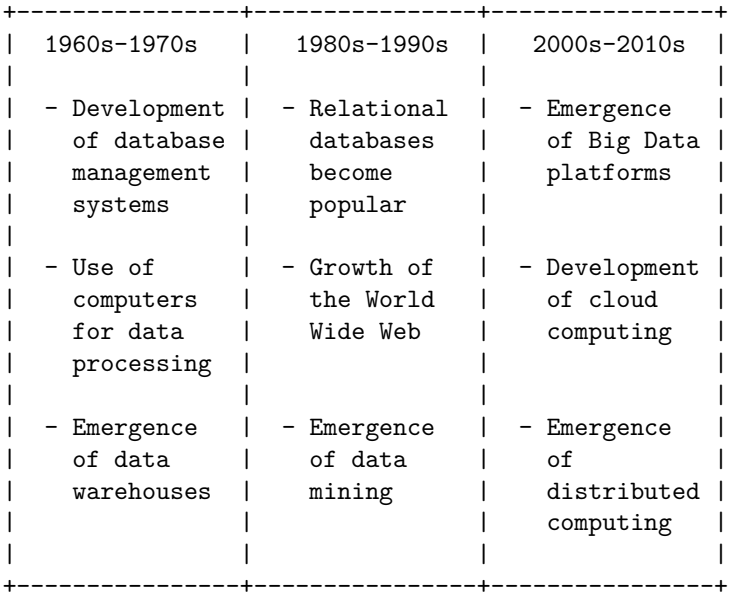

Characteristics
Sources
Storage
Processing
Analysis

Types of digital data in big data



Digital data in big data can be categorized into three types: structured, unstructured, and semi-structured. Structured data refers to data that is organized into a specific format or schema, such as a database. Unstructured data refers to data that does not have a specific format or structure, such as text or images. Semi-structured data is a combination of both structured and unstructured data, and can include data such as emails or XML files, which have some structure but are not as rigidly organized as a database.

History of Big Data Innovation



Introduction to Big Data Platform

Big Data refers to the large and complex data sets that are difficult to process using traditional data processing applications. These data sets are characterized by the 3Vs: Volume, Velocity, and Variety.

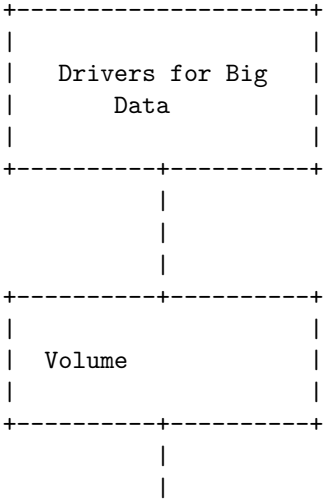
A Big Data platform is a type of IT solution that combines the features and capabilities of several Big Data application and tools to provide an organization with a comprehensive solution for managing, processing, and analyzing large and complex data sets.

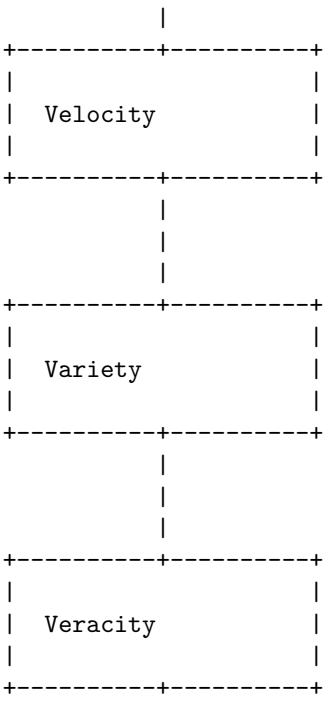
Some of the key features of a Big Data platform include: - Data storage and management: A Big Data platform provides the ability to store and manage large volumes of structured, semi-structured, and unstructured data. - Data processing: A Big Data platform provides the ability to process large and complex data sets using distributed computing techniques. - Data analysis: A Big Data platform provides the ability to analyze large and complex data sets using advanced analytics techniques such as machine learning and data mining. - Data visualization: A Big Data platform provides the ability to visualize large and complex data sets using data visualization tools.

Some of the popular Big Data platforms include Hadoop, Spark, and NoSQL databases. These platforms provide a scalable and flexible solution for managing, processing, and analyzing large and complex data sets.

In summary, a Big Data platform is a comprehensive IT solution that provides organizations with the ability to manage, process, and analyze large and complex data sets. These platforms are essential for organizations that need to derive insights and value from their data.

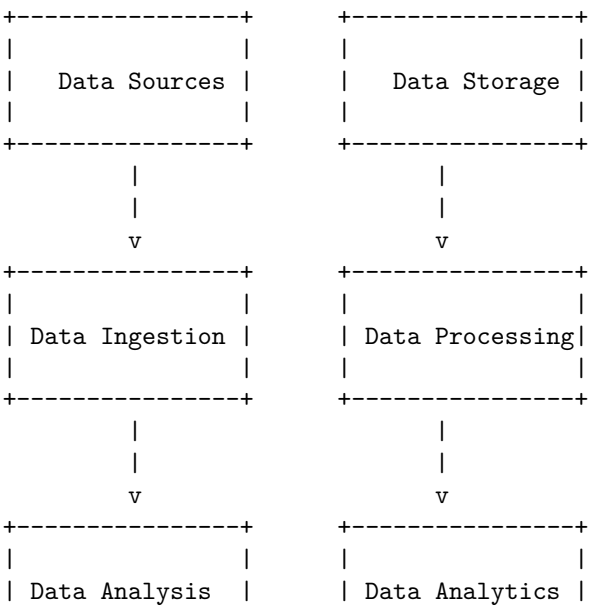
Drivers for Big Data





Big Data Architecture

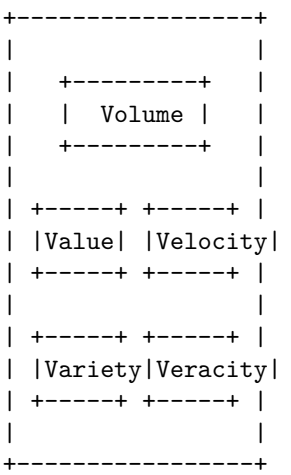
Here is an ASCII diagram of a typical Big Data Architecture:



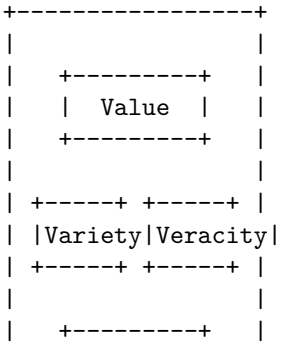
This diagram shows the flow of data from its sources, through ingestion and processing, to analysis and analytics. The data sources can be anything from databases, log files, and social media feeds, to sensors and other IoT devices. The data is ingested and stored in a data storage system, which can be a traditional relational database, a NoSQL database, or a data lake. The data is then processed, either in batch or in real-time, to extract insights and make it ready for analysis. Finally, the data is analyzed and visualized using various analytics tools and techniques.

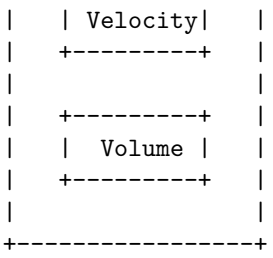
Big data is characterized by the 5 Vs: Volume, Velocity, Variety, Veracity, and Value. Here is an ASCII diagram that represents these characteristics:

Big data characteristics



The 5 Vs of Big Data refer to the five key characteristics that define Big Data: Volume, Velocity, Variety, Veracity, and Value. Here is an ASCII diagram that represents the 5 Vs of Big Data:



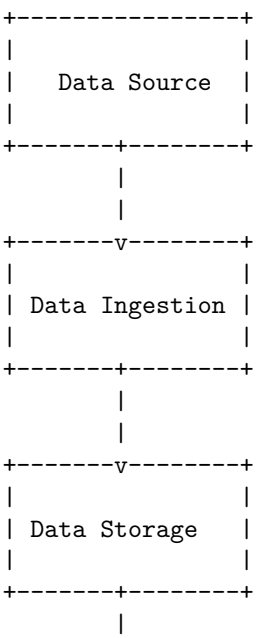


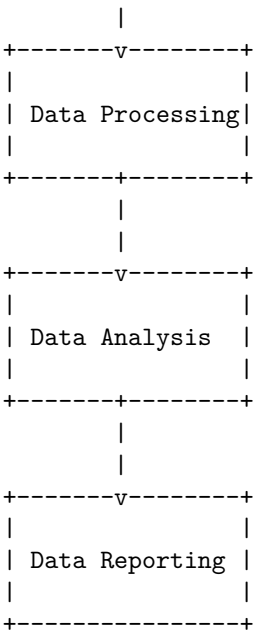
5 Vs of Big Data

1. **Volume**: Refers to the vast amount of data generated every second. This data comes from various sources such as social media, business transactions, and sensors, among others.
2. **Velocity**: Refers to the speed at which data is generated and processed. The faster the data is generated and processed, the more valuable it is.
3. **Variety**: Refers to the different types of data that are generated. This includes structured data, unstructured data, and semi-structured data.
4. **Veracity**: Refers to the quality and accuracy of the data. Inaccurate or low-quality data can lead to incorrect conclusions and decisions.
5. **Value**: Refers to the ability to extract useful insights from the data. The more valuable the insights, the more valuable the data.

Big Data technology components

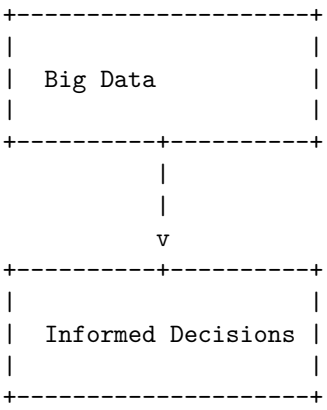
Here is an ASCII diagram of the Big Data technology components:





Big Data importance

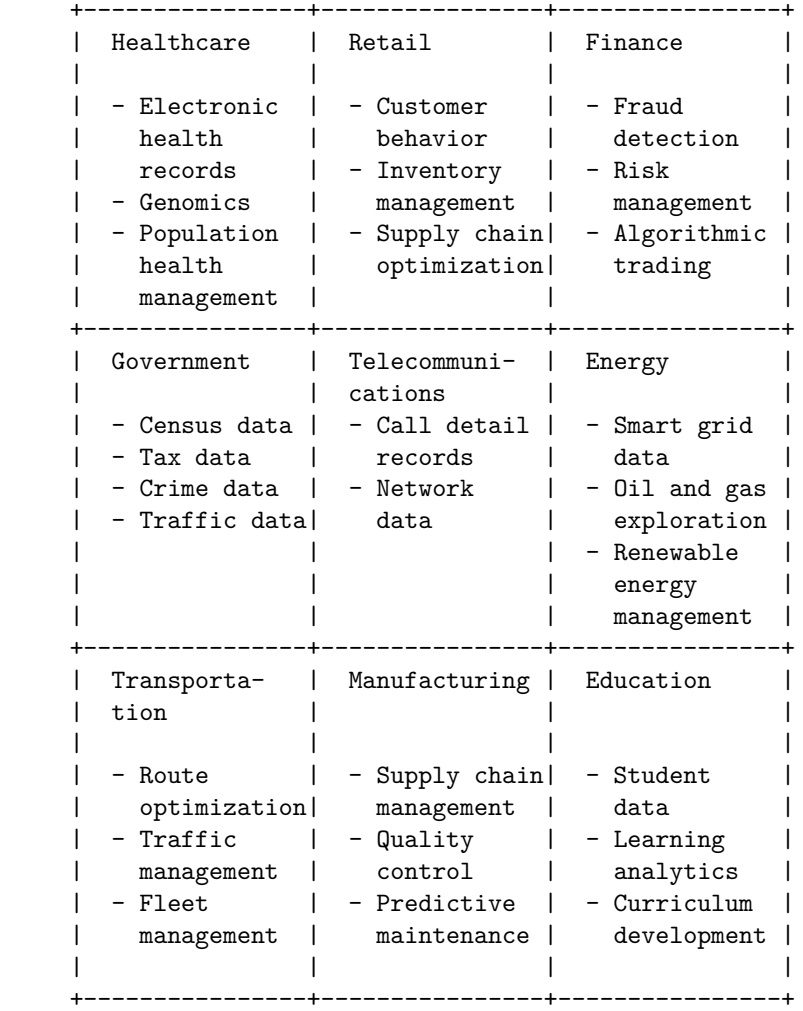
Big Data is important because it allows organizations to make more informed decisions by analyzing large and complex data sets. Here is an ASCII diagram that illustrates the importance of Big Data:



This diagram shows that Big Data is the input that leads to informed decisions as the output. By analyzing large and complex data sets, organizations can gain insights that would not be possible with smaller data sets. This can lead to better decision-making, improved efficiency, and increased competitiveness.

Big Data applications

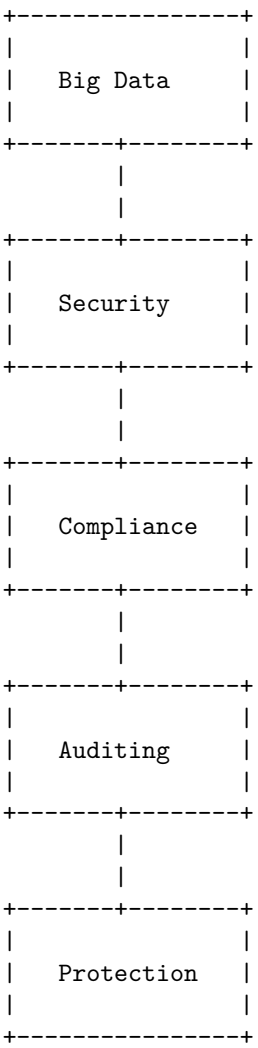
Here is an ASCII diagram that shows some common applications of Big Data:



This diagram shows some common applications of Big Data across various industries, including healthcare, retail, finance, government, telecommunications, energy, transportation, manufacturing, and education. Within each industry, specific use cases are listed, such as electronic health records in healthcare, customer behavior in retail, and fraud detection in finance.

Big Data features – security, compliance, auditing and protection

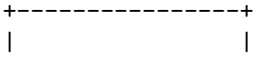
Here is an ASCII diagram that illustrates the relationship between security, compliance, auditing, and protection in the context of Big Data:

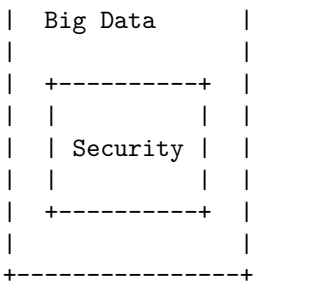


In the context of Big Data, security refers to the measures taken to protect data from unauthorized access or theft. Compliance refers to the adherence to laws, regulations, and standards that govern the collection, storage, and use of data. Auditing refers to the process of reviewing and verifying that the data is being handled in accordance with the established policies and procedures. Protection refers to the measures taken to ensure the integrity and availability of the data.

Security of Big Data

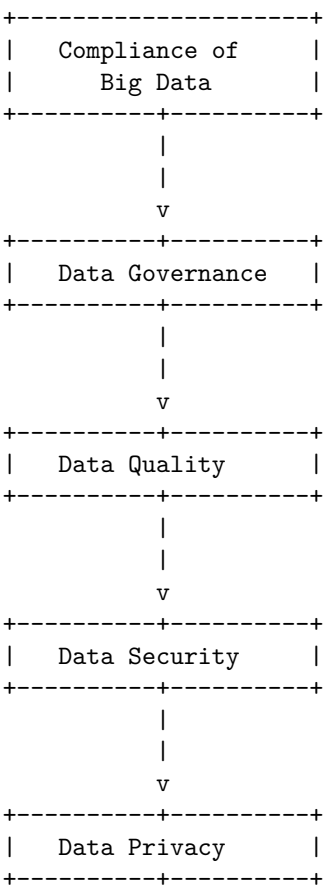
Here is an ASCII diagram that represents the security of Big Data:





Compliance of Big Data

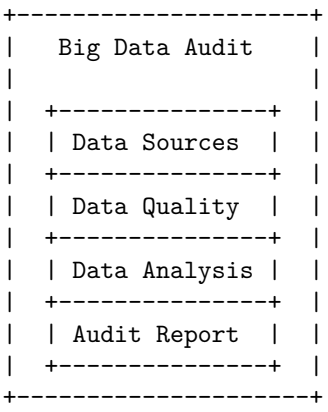
Here is an ASCII diagram that represents the compliance of Big Data:



This diagram shows the different aspects of compliance when it comes to Big Data. The first step is Data Governance, which involves the overall management of the availability, usability, integrity, and security of the data used in an organization. The next step is Data Quality, which ensures that the data is

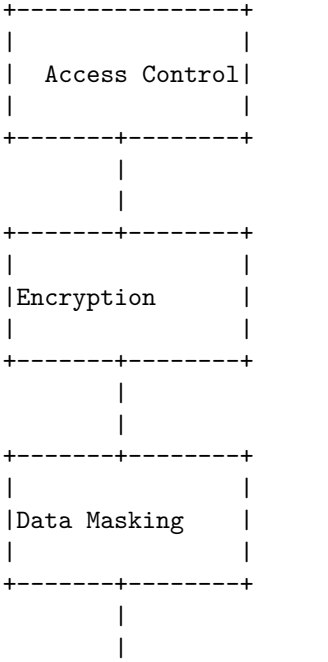
accurate, complete, consistent, and reliable. Data Security involves protecting the data from unauthorized access, use, disclosure, disruption, modification, or destruction. Finally, Data Privacy involves ensuring that the data is collected, stored, and used in compliance with relevant laws and regulations.

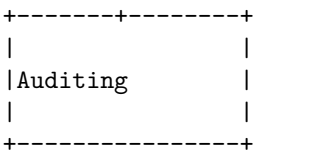
Auditing of Big Data



Protection of Big Data

Here is an ASCII diagram that illustrates some of the measures that can be taken to protect Big Data:

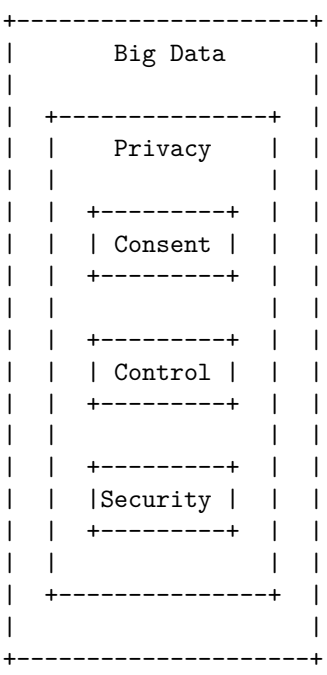




This diagram shows that there are several layers of protection that can be applied to Big Data. Access control is the first line of defense, which ensures that only authorized users can access the data. Encryption is another important measure that can be used to protect the data while it is being transmitted or stored. Data masking is a technique that can be used to hide sensitive information from unauthorized users. Finally, auditing is a process that can be used to monitor and track access to the data, to ensure that it is being used appropriately.

Big Data privacy

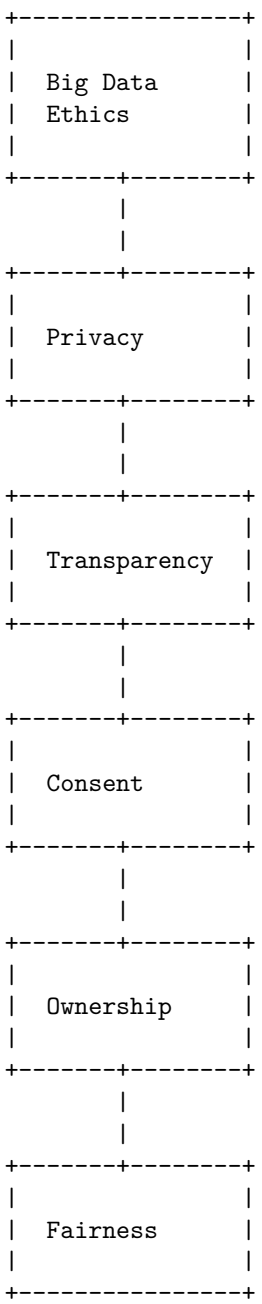
Here is an ASCII diagram that represents the concept of Big Data privacy:



This diagram shows that privacy is an important aspect of Big Data. Within the privacy component, there are several sub-components such as consent, control, and security. Consent refers to the user's agreement to the collection and use of their data. Control refers to the user's ability to manage their data and how it is used. Security refers to the measures taken to protect the user's data from unauthorized access or misuse.

Big Data ethics

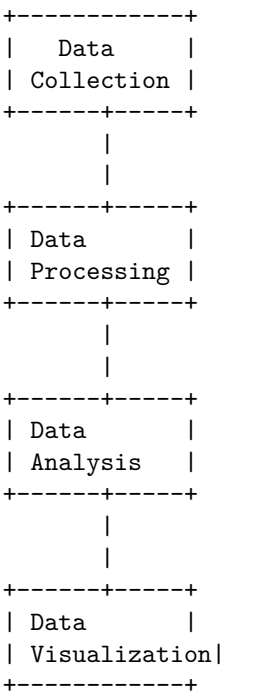
Here is an ASCII diagram that represents some of the key ethical considerations in Big Data:



This diagram shows that some of the key ethical considerations in Big Data include privacy, transparency, consent, ownership, and fairness. These are important factors to consider when collecting, storing, and analyzing large amounts of data.

Big Data Analytics

Here is an ASCII diagram that represents the process of Big Data Analytics:



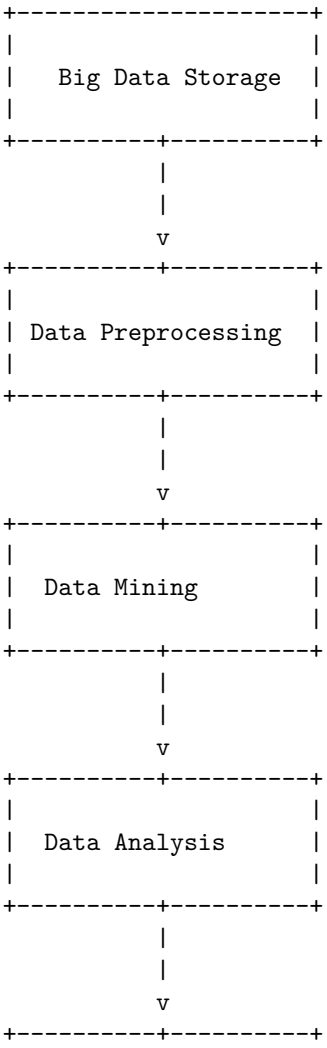
The first step in Big Data Analytics is **Data Collection**, where data is gathered from various sources. The collected data is then **processed** to clean, transform, and organize it for analysis. After processing, the data is **analyzed** to discover patterns, relationships, and trends. Finally, the results of the analysis are **visualized** to communicate insights and facilitate decision-making.

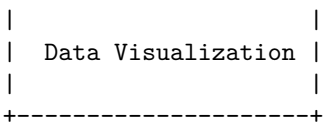
Challenges of conventional systems compared to Big Data

Conventional Systems	Big Data
Limited scalability	High scalability
Limited storage	High storage capacity

Limited processing power	High processing power
Structured data only	Structured and unstructured data
Limited data sources	Multiple data sources
Batch processing	Real-time processing

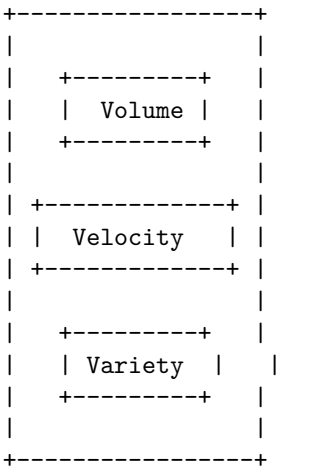
Intelligent Data Analysis in Big Data





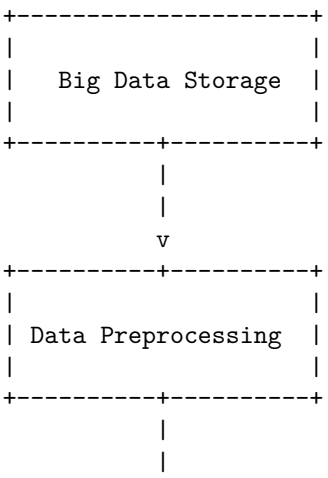
Nature of Data in Big Data

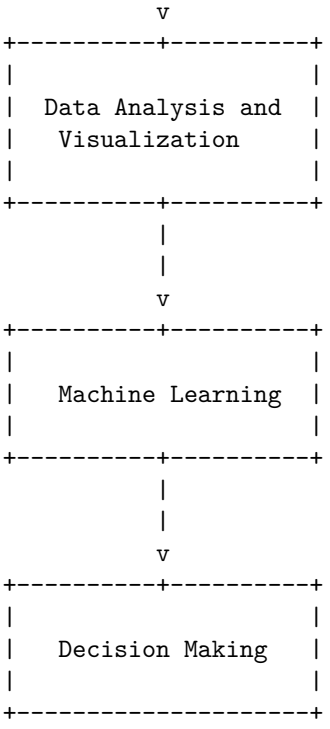
Big Data is characterized by the 3 Vs: Volume, Velocity, and Variety.



Volume refers to the large amount of data that is generated and stored. Velocity refers to the speed at which data is generated, processed, and analyzed. Variety refers to the different types of data, both structured and unstructured, that are generated and need to be processed.

Analytic Processes and Tools for Big Data

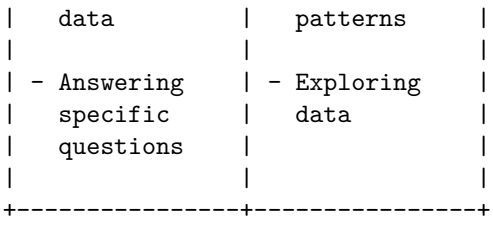




This diagram shows the analytic processes and tools for Big Data. The first step is to store the Big Data in a suitable storage system. Then, the data is preprocessed to clean and transform it into a usable format. After preprocessing, the data is analyzed and visualized to extract insights and patterns. Machine learning algorithms can then be applied to the data to make predictions and decisions. Finally, the insights and predictions are used to make informed decisions.

Analysis vs Reporting in Big Data

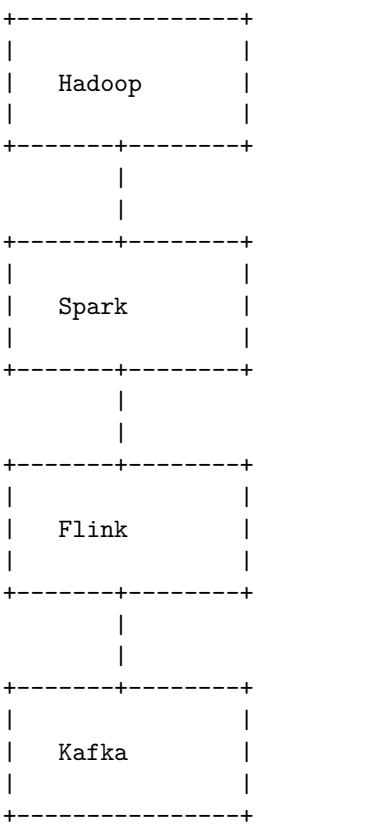
Reporting	Analysis
- Historical data	- Predictive modeling
- Summarizing data	- Data mining
- Presenting	- Identifying



Reporting and analysis are two different aspects of Big Data. Reporting is focused on summarizing and presenting historical data to answer specific questions. Analysis, on the other hand, is focused on exploring data, identifying patterns, and using predictive modeling and data mining techniques to gain insights and make predictions.

Modern Data Analytic Tools for Big Data

Here is an ASCII diagram that shows some of the modern data analytic tools for Big Data:

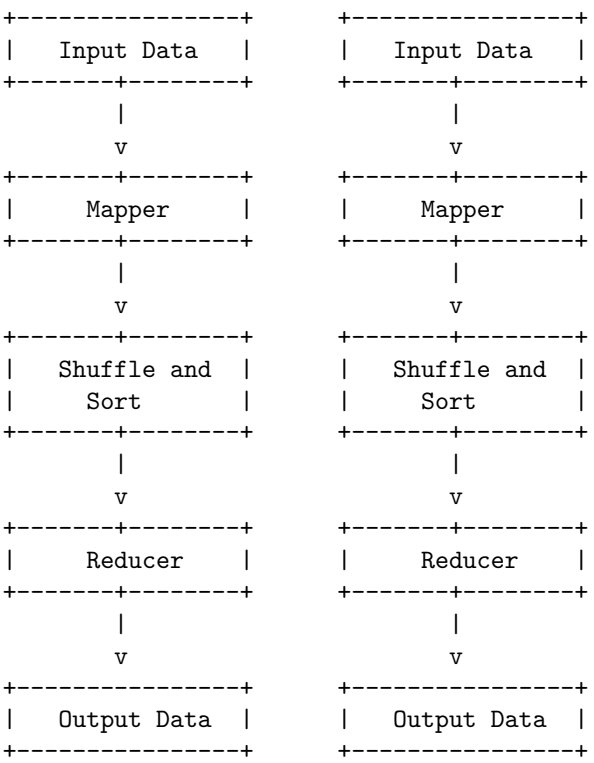


These tools are commonly used for processing and analyzing large datasets.

Hadoop is a framework for distributed storage and processing of large datasets. Spark is a fast and general-purpose cluster computing system. Flink is a stream processing framework. Kafka is a distributed streaming platform.

Unit 2 - Hadoop and Map Reduce

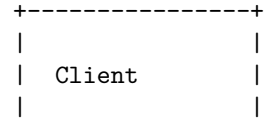
Here is an ASCII diagram that illustrates the basic architecture of Hadoop and MapReduce:

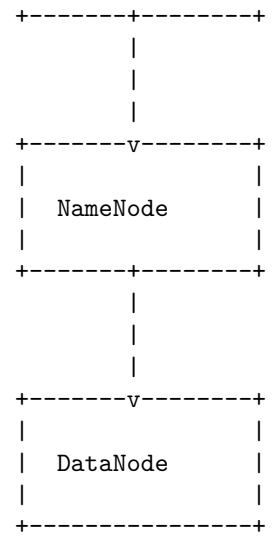


In this diagram, the input data is first split into multiple chunks and fed into the mappers. The mappers then process the data and generate intermediate key-value pairs. These key-value pairs are then shuffled and sorted before being fed into the reducers. The reducers then aggregate the data based on the keys and generate the final output.

Hadoop

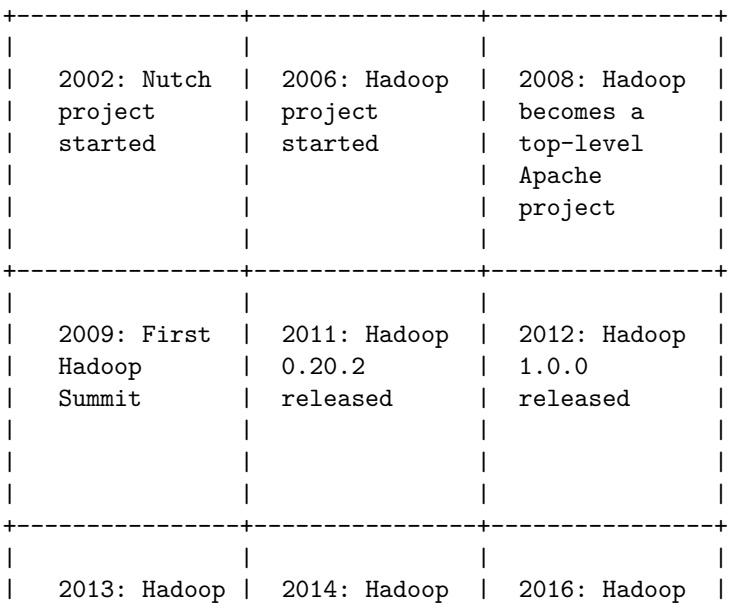
Here is an ASCII diagram of the Hadoop architecture:

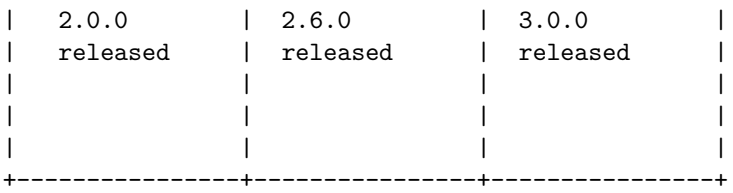




The diagram shows the basic architecture of Hadoop, which consists of a Client, a NameNode, and a DataNode. The Client communicates with the NameNode to access data stored on the DataNode. The NameNode manages the file system namespace and regulates access to files by clients. The DataNode stores data in the Hadoop Distributed File System (HDFS) and serves read and write requests from the file system's clients.

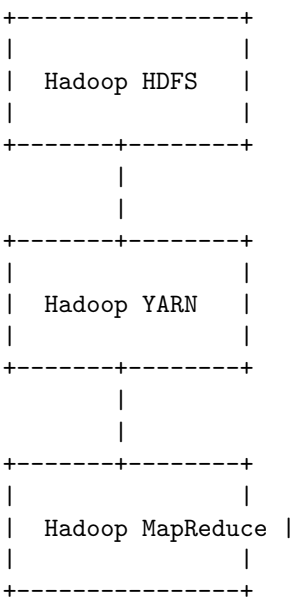
History of Hadoop





Apache Hadoop

Here is an ASCII diagram of Apache Hadoop:



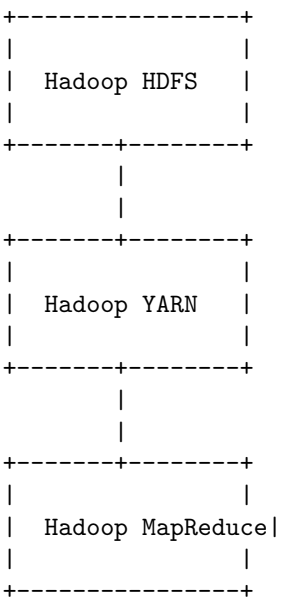
Hadoop HDFS is the storage layer of Apache Hadoop, which stores data in a distributed manner. Hadoop YARN is the resource management layer, which manages the allocation of resources for processing data. Hadoop MapReduce is the processing layer, which processes data in a distributed manner using the MapReduce programming model.

Hadoop Distributed File System

- The Hadoop Distributed File System (HDFS) is the primary data storage system used by Hadoop applications.
- HDFS employs a NameNode and DataNode architecture to implement a distributed file system that provides high-performance access to data across highly scalable Hadoop clusters.
- The core of Apache Hadoop consists of a storage part, known as Hadoop Distributed File System (HDFS), and a processing part which is a MapReduce programming model.

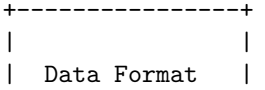
- Hadoop splits files into large blocks and distributes them across nodes in a cluster. It then transfers packaged code into nodes to process the data in parallel.
- As the primary component of the Hadoop ecosystem, HDFS is a distributed file system that provides high-throughput access to application data with no need for schemas to be defined up front.
- HDFS is a distributed file system that handles large data sets running on commodity hardware. It is used to scale a single Apache Hadoop cluster to hundreds (and even thousands) of nodes.
- HDFS is one of the major components of Apache Hadoop, the others being MapReduce and YARN.

Components of Hadoop



Hadoop is an open-source software framework for storing and processing big data in a distributed fashion. It consists of several components, including Hadoop Distributed File System (HDFS), Hadoop YARN, and Hadoop MapReduce. HDFS is the primary storage system used by Hadoop applications, while YARN is the resource management layer that schedules and manages the resources used by the applications. MapReduce is the programming model used to process large data sets in parallel across a Hadoop cluster.

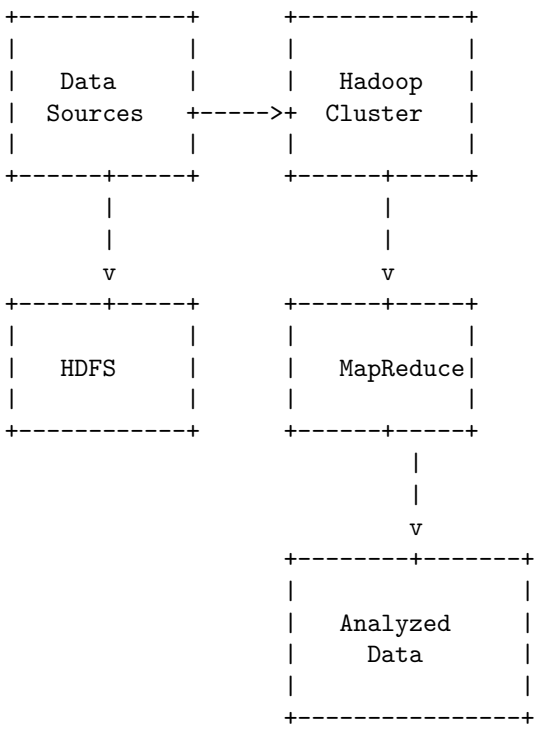
Data Format Co



```
|       Co       |
|                |
+----------------+
```

Analyzing data with Hadoop

Here is an ASCII diagram that shows the process of analyzing data with Hadoop:

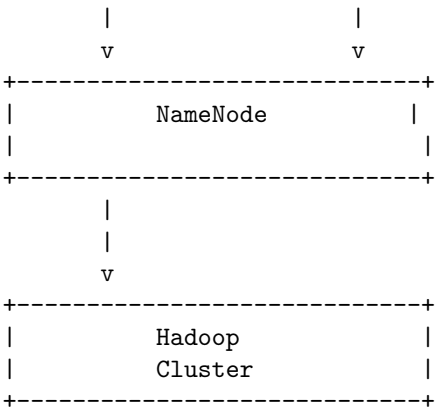


Hadoop is an open-source software framework that allows for the distributed processing of large data sets across clusters of computers. Data from various sources is first ingested into the Hadoop cluster and stored in the Hadoop Distributed File System (HDFS). MapReduce, a programming model for processing large data sets, is then used to analyze the data. The output is the analyzed data, which can be used for further analysis or reporting.

Scaling out with Hadoop

Here is an ASCII diagram that illustrates how scaling out with Hadoop works:

```
+------------+     +------------+
|   DataNode |     |   DataNode |
|            |     |            |
+------------+     +------------+
       |                 |
```



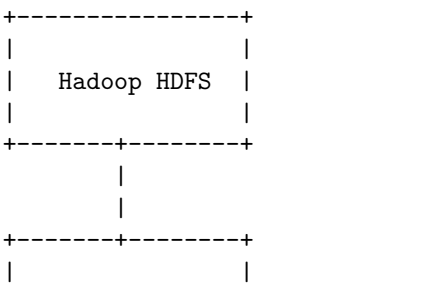
Hadoop Streaming

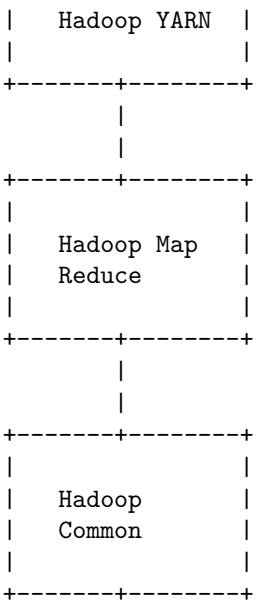
- Hadoop streaming is a utility that comes with the Hadoop distribution .
- The utility allows you to create and run Map/Reduce jobs with any executable or script as the mapper and/or the reducer .
- You can supply a Java class as the mapper and/or the reducer .
- Hadoop streaming enables us to create or run MapReduce scripts in any language either, java or non-java, as mapper/reducer .
- Hadoop streaming is a powerful feature that allows anyone to write their code in any language of their own choice .

Hadoop Pipes

- Hadoop Pipes is the name of the C++ interface to Hadoop MapReduce.
- Unlike Streaming, which uses standard input and output to communicate with the map and reduce code, Pipes uses sockets as the channel over which the tasktracker communicates with the process running the C++ map or reduce function.
- JNI is not used.
- Hadoop Pipes uses sockets to enable tasktrackers to communicate processes running the C++ map or reduce functions.

Hadoop Echo System

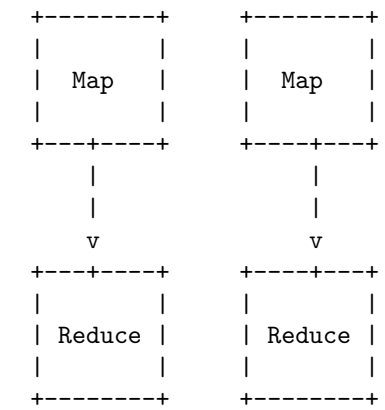




Map Reduce

MapReduce is a programming model for processing large data sets with a parallel, distributed algorithm on a cluster. The model is inspired by the map and reduce functions commonly used in functional programming, although their purpose in the context of MapReduce is not the same.

Here is an ASCII diagram of the MapReduce process:



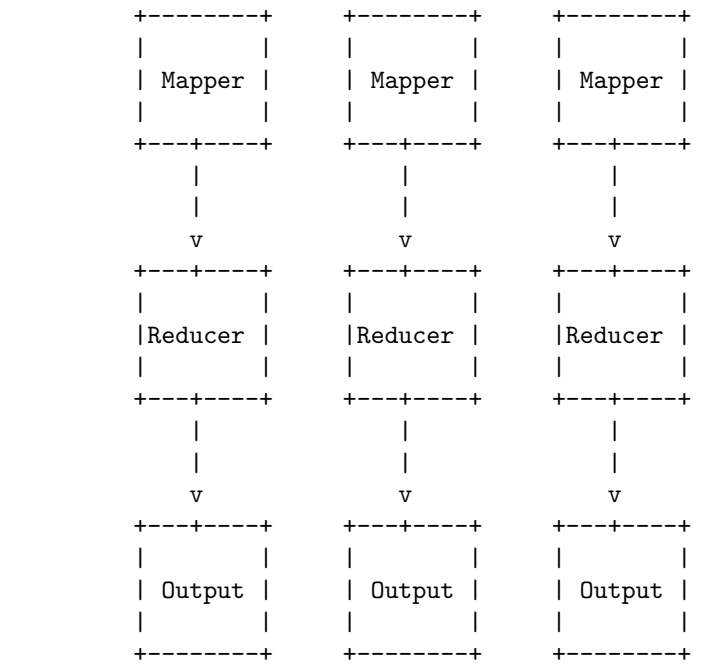
In the MapReduce model, the data processing primitives are called mappers and reducers. The input data is split into independent chunks that are processed by the map tasks in a completely parallel manner. The framework sorts the outputs of the maps, which are then input to the reduce tasks. Typically both

the input and the output of the job are stored in a distributed file system. The framework takes care of scheduling tasks, monitoring them and re-executing the failed tasks.

Map Reduce framework and basics

MapReduce is a programming model and an associated implementation for processing and generating large data sets. The model is inspired by the map and reduce functions commonly used in functional programming, although their purpose in the MapReduce framework is not the same as their original forms.

Here is an ASCII diagram that illustrates the basic flow of data in a MapReduce job:



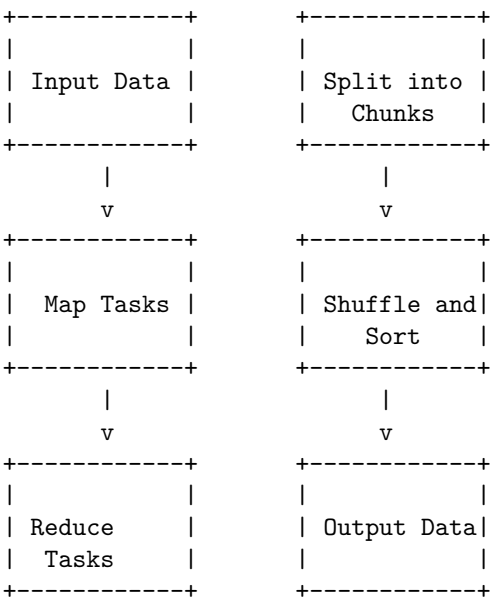
In this diagram, the input data is split into multiple chunks and processed by multiple mapper tasks in parallel. The output of the mappers is then shuffled and sorted, and fed into the reducers. The reducers process the data and generate the final output.

MapReduce is a programming model and an associated implementation for processing and generating large data sets. It works by dividing the input data into multiple chunks, which are processed independently by different worker nodes in parallel. The results of these computations are then combined to produce the final output.

Here is an ASCII diagram that illustrates how MapReduce works:

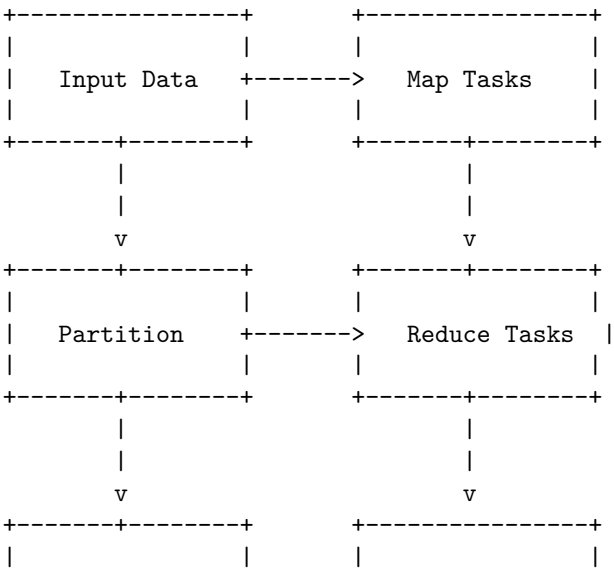
```
Input Data -> Split into Chunks -> Map Tasks -> Shuffle and Sort -> Reduce Tasks -> Output D
```

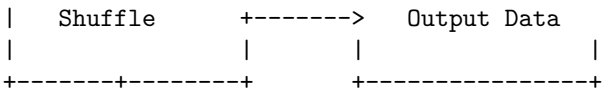
How MapReduce Works



Developing a Map Reduce application

Here is an ASCII diagram that illustrates the process of developing a Map Reduce application:

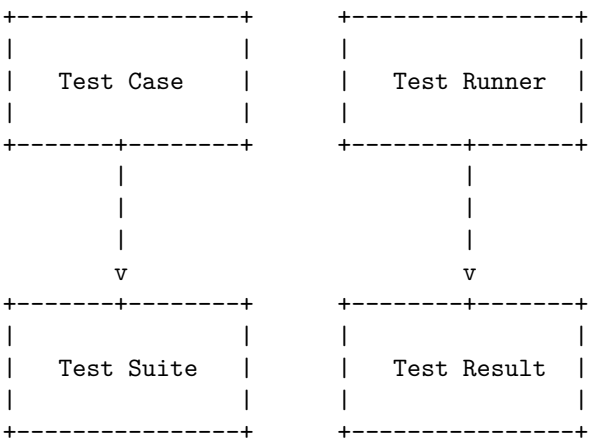




The process of developing a Map Reduce application involves several steps. First, the input data is fed into the Map tasks, which process the data and generate intermediate key-value pairs. These key-value pairs are then partitioned and shuffled to the Reduce tasks, which aggregate the data and generate the final output. The output data is then written to the specified location.

Unit Tests with MR Unit

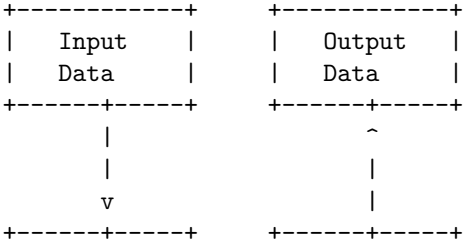
Here is an ASCII diagram that represents the process of unit testing with MR Unit:

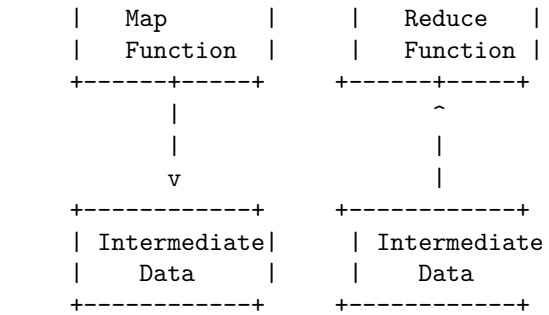


In this diagram, the test case is the individual unit test that is written to test a specific functionality. The test suite is a collection of test cases that are run together. The test runner is the tool that runs the test suite and generates the test result, which shows whether the tests passed or failed.

Test Data and Local Tests in Map Reduce

Here is an ASCII diagram that illustrates the process of testing data and running local tests in a MapReduce framework:





In this diagram, the input data is processed by the Map function, which generates intermediate data. The intermediate data is then processed by the Reduce function, which generates the final output data. This process can be tested locally by running the Map and Reduce functions on a smaller set of test data to ensure that the functions are working as expected.

Anatomy of a MapReduce Job Run

MapReduce is a programming model for processing large data sets with a parallel, distributed algorithm on a cluster. A MapReduce job usually splits the input data into independent chunks, which are processed by the map tasks in a completely parallel manner. The framework sorts the outputs of the maps, which are then input to the reduce tasks. Typically, both the input and the output of the job are stored in a distributed file system.

The anatomy of a MapReduce job run can be broken down into the following steps:

1. **Job Submission:** The user submits the job to the MapReduce framework by specifying the input and output locations, the map and reduce functions, and other job-specific parameters.
2. **Input Splitting:** The input data is divided into splits, which are logical chunks of the input data. Each split is assigned to a map task.
3. **Scheduling:** The MapReduce framework schedules the map and reduce tasks on the available nodes in the cluster.
4. **Map Task Execution:** Each map task reads its input split and applies the user-defined map function to each record. The output of the map function is written to the local disk.
5. **Shuffle and Sort:** The MapReduce framework collects the output of the map tasks and sorts it by key. The sorted data is then shuffled across the network to the nodes where the reduce tasks are scheduled to run.
6. **Reduce Task Execution:** Each reduce task reads the shuffled data and applies the user-defined reduce function to the values associated with each

key. The output of the reduce function is written to the distributed file system.

7. **Job Completion:** The MapReduce framework notifies the user when the job is complete and provides status and performance information.

This is a high-level overview of the anatomy of a MapReduce job run. Each step in the process can be further broken down and optimized for specific use cases and data sets.

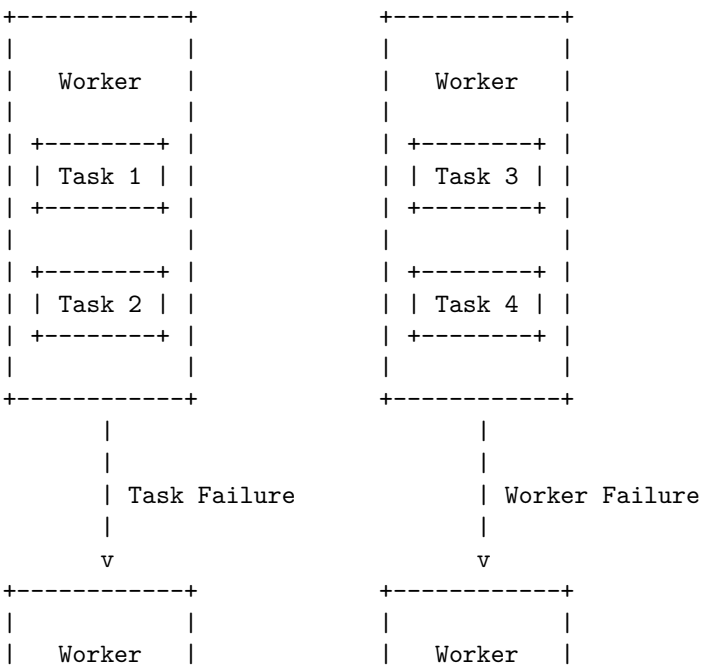
Failures in MapReduce

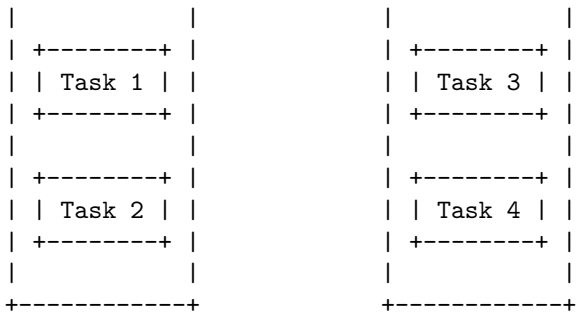
In a MapReduce system, there are two types of failures that can occur: Task Failure and Worker Failure.

Task Failure occurs when a task fails to complete successfully. This can happen for a variety of reasons, such as a bug in the code or a problem with the input data. When a task fails, the MapReduce system will automatically reassign the task to another worker to be re-executed.

Worker Failure occurs when a worker node fails. This can happen due to hardware or software issues on the worker node. When a worker node fails, the MapReduce system will automatically reassign any tasks that were in progress on the failed worker to other workers to be re-executed.

Here is an ASCII diagram that illustrates these two types of failures in a MapReduce system:

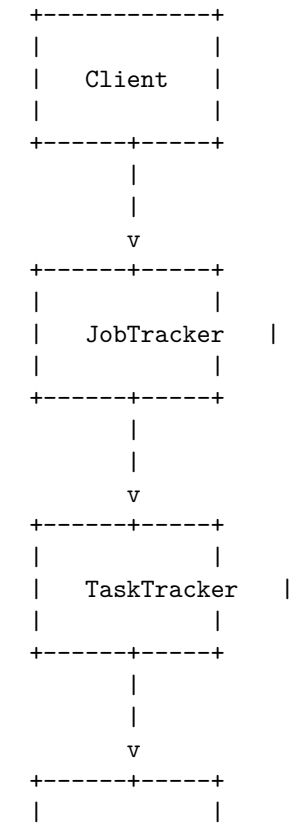




In the diagram above, Task 2 on the left worker fails and is reassigned to the right worker. The left worker then experiences a Worker Failure and all of its tasks (Task 1 and Task 2) are reassigned to the right worker. The right worker then executes all four tasks (Task 1, Task 2, Task 3, and Task 4).

Job scheduling in MapReduce

Here is an ASCII diagram that illustrates the process of job scheduling in MapReduce:



```
|   DataNode   |
|              |
+------+-----+
```

In this diagram, the client submits a job to the JobTracker, which is responsible for scheduling the job and assigning tasks to TaskTrackers. The TaskTrackers then execute the tasks and communicate with the DataNodes to read and write data.

Shuffle and Sort in MapReduce

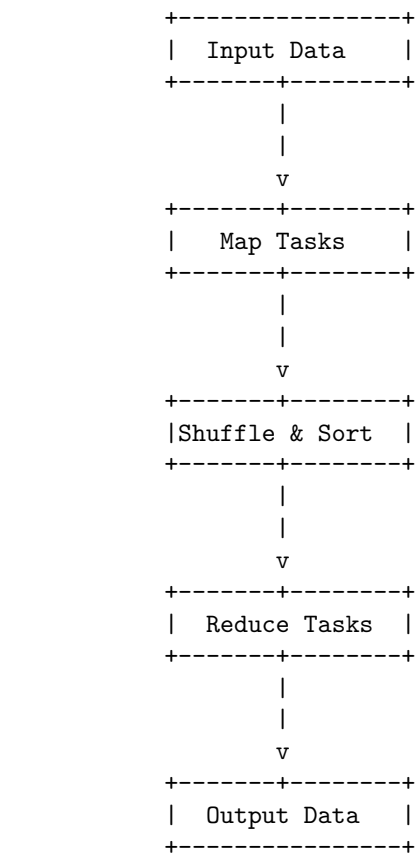
The shuffle and sort phase in MapReduce is the process of transferring data from the mappers to the reducers. Here is an ASCII diagram that illustrates the process:

```
+------------+
|  Mapper 1  |
+------+-----+
       |
       v
+------+-----+
|  Mapper 2  |
+------+-----+
       |
       v
+------+-----+
|  Mapper 3  |
+------+-----+
       |
       v
+------+-----+
|  Shuffle   |
+------+-----+
       |
       v
+------+-----+
|   Sort     |
+------+-----+
       |
       v
+------+-----+
|  Reducer 1 |
+------+-----+
       |
       v
+------+-----+
|  Reducer 2 |
```

```
          +------+-----+
```

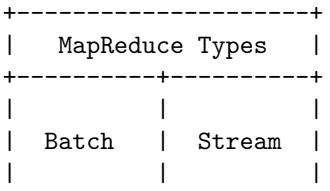
Task Execution in Map Reduce

Here is an ASCII diagram that illustrates the task execution in MapReduce:



In MapReduce, the input data is first divided into splits and assigned to map tasks. The map tasks process the data and generate intermediate key-value pairs. These key-value pairs are then shuffled and sorted, and assigned to reduce tasks. The reduce tasks process the data and generate the final output.

Map Reduce types in map reduce



```
+----------+----------+
```

MapReduce is a programming model for processing large data sets with a parallel, distributed algorithm on a cluster. There are two main types of MapReduce: Batch and Stream.

Batch MapReduce is used for processing large amounts of static data, where the data is divided into chunks and processed in parallel by multiple machines. The results are then combined to produce the final output.

Stream MapReduce, on the other hand, is used for processing data in real-time as it is generated. The data is processed in parallel by multiple machines as it arrives, and the results are continuously updated.

Input Formats in MapReduce

Here is an ASCII diagram that illustrates the input formats in MapReduce:

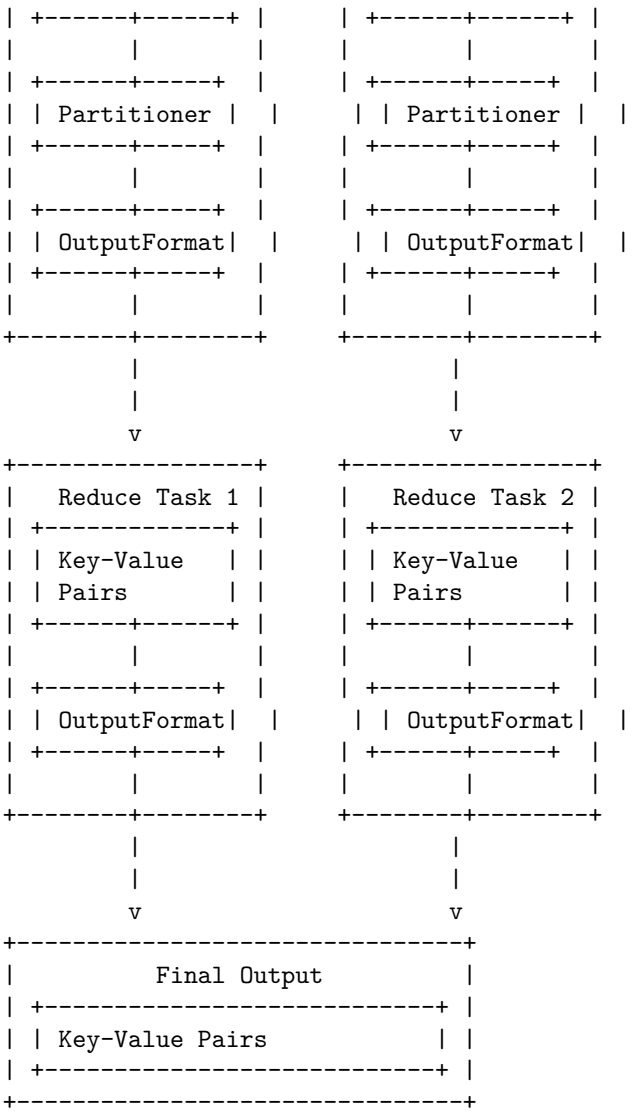
```
+----------------+     +----------------+
|                |     |                |
|  InputFormat   |     |  InputSplit    |
|                |     |                |
+-------+--------+     +--------+-------+
        |                       |
        |                       |
        |                       |
        |                       |
        |                       |
        |                       |
+-------+--------+     +--------+-------+
|                |     |                |
|  RecordReader  |     |  Mapper        |
|                |     |                |
+----------------+     +----------------+
```

In MapReduce, an `InputFormat` is responsible for defining how input data is split into `InputSplits` and how these splits are read by the `RecordReader`. The `RecordReader` reads the data from the `InputSplit` and converts it into key-value pairs that are fed into the `Mapper` for processing.

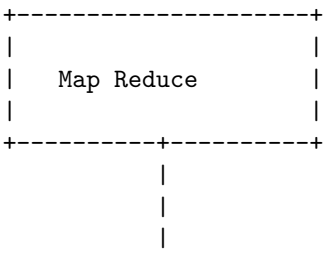
Output Formats in Map Reduce

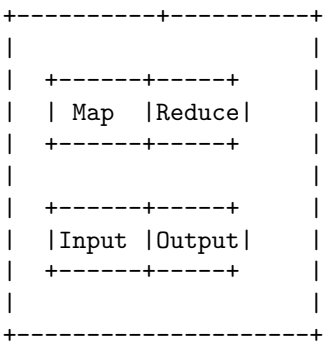
Here is an ASCII diagram that shows the output formats in Map Reduce:

```
+-----------------+     +-----------------+
|   Map Task 1    |     |   Map Task 2    |
| +-------------+ |     | +-------------+ |
| | Key-Value   | |     | | Key-Value   | |
| | Pairs       | |     | | Pairs       | |
```



Map Reduce features





Real-world Map Reduce

MapReduce is a programming model and an associated implementation for processing and generating large data sets. It is designed to handle large volumes of data in parallel by dividing the work into a set of independent tasks.

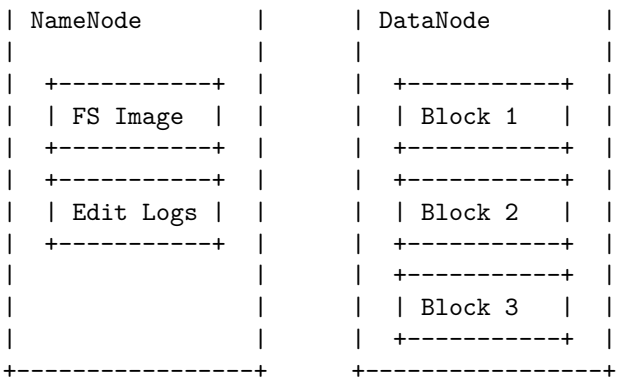
Some real-world applications of MapReduce include:

1. **Large scale data processing**: MapReduce can be used to process large amounts of data in parallel, making it ideal for tasks such as log analysis, data mining, and web indexing.
2. **Distributed computing**: MapReduce can be used to distribute computation across a large number of machines, allowing for the processing of large data sets that would be difficult to handle on a single machine.
3. **Machine learning**: MapReduce can be used to implement machine learning algorithms, such as clustering and classification, on large data sets.
4. **Data filtering and transformation**: MapReduce can be used to filter and transform large data sets, such as converting data from one format to another or removing unwanted data.
5. **Graph processing**: MapReduce can be used to process large graphs, such as social networks or web graphs, to find patterns and relationships between nodes.

Overall, MapReduce is a powerful tool for processing large data sets in a distributed and parallel manner, making it well-suited for a wide range of real-world applications.

Unit 4 - HDFS (Hadoop Distributed File System) and Hadoop Environment

Here is an ASCII diagram of the Hadoop Distributed File System (HDFS) and Hadoop Environment:



The NameNode is the master node in the Hadoop Distributed File System (HDFS) and is responsible for managing the file system namespace and regulating access to files by clients. The NameNode stores the metadata for the file system, including the file system tree and the mapping of blocks to DataNodes. The FS Image is a file that contains the entire file system namespace, while the Edit Logs record changes to the file system.

The DataNode is a slave node in the Hadoop Distributed File System (HDFS) and is responsible for storing the data blocks of files. Each DataNode stores a set of blocks and periodically sends a report of all the blocks it is storing to the NameNode. The NameNode uses this information to ensure that the data is replicated across multiple DataNodes for fault tolerance.

HDFS

HDFS stands for Hadoop Distributed File System. It is a distributed file system that handles large data sets running on commodity hardware. It is used to scale a single Apache Hadoop cluster to hundreds (and even thousands) of nodes . HDFS is one of the major components of Apache Hadoop, the others being MapReduce and YARN .

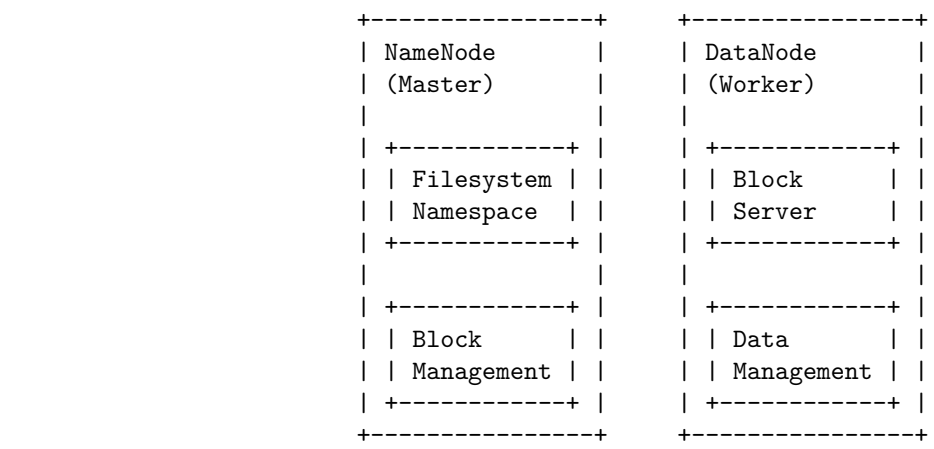
HDFS is fault-tolerant and designed to be deployed on low-cost, commodity hardware . It is the primary storage system used by Hadoop applications. This open-source framework works by rapidly transferring data between nodes. It's often used by companies who need to handle and store big data .

HDFS has many similarities with existing distributed file systems. However, the differences from other distributed file systems are significant. HDFS is highly fault-tolerant and is designed to be deployed on low-cost hardware .

Design of HDFS

HDFS is designed to store and manage very large files across multiple machines. It is based on the principle of data locality, which means that data is stored on

the same machine where it is processed. Here is an ASCII diagram of the design of HDFS:



The NameNode is the master server that manages the file system namespace and regulates access to files by clients. The DataNodes are worker nodes that store and manage the data blocks. The NameNode and DataNodes communicate with each other to ensure that data is stored and retrieved reliably.

HDFS Concepts

HDFS (Hadoop Distributed File System) is a distributed file system designed to run on commodity hardware. It is highly fault-tolerant and is designed to be deployed on low-cost hardware. Some of the key concepts of HDFS are:

1. **NameNode and DataNode**: HDFS has a master/slave architecture. The NameNode is the master server that manages the file system namespace and regulates access to files by clients. The DataNodes are the slave servers that manage the storage attached to the nodes that they run on.
2. **Block Size**: HDFS stores large files as a sequence of blocks. The default block size is 64 MB, but it can be configured to a larger size.
3. **Replication**: HDFS replicates each block of data on multiple DataNodes to ensure high availability and fault tolerance. The default replication factor is 3, but it can be configured to a different value.
4. **Rack Awareness**: HDFS is designed to be aware of the network topology of the cluster. It tries to place replicas of data blocks on different racks to improve data reliability and reduce the impact of rack failure.
5. **Data Pipelining**: When a client writes data to HDFS, the data is first written to a local buffer. When the buffer reaches a certain size, the data is sent to the first DataNode in the pipeline. The first DataNode then forwards the data to the second DataNode in the pipeline, and so

on. This improves write performance and reduces the impact of network latency.

6. **Data Locality**: HDFS tries to schedule tasks on the same node where the data is stored, or as close as possible. This reduces the amount of data that needs to be transferred over the network and improves performance.
7. **Federation**: HDFS supports federation, which allows multiple independent NameNodes to share the same physical storage. This improves scalability and allows multiple namespaces to coexist within the same cluster.
8. **High Availability**: HDFS supports high availability through the use of an active and a standby NameNode. If the active NameNode fails, the standby NameNode takes over its duties to ensure that the file system remains available.

These are some of the key concepts of HDFS. It is a powerful and flexible distributed file system that is widely used in big data processing and analytics.

Hadoop Distributed File System (HDFS) is a distributed file system designed to run on commodity hardware. It has many benefits, including:

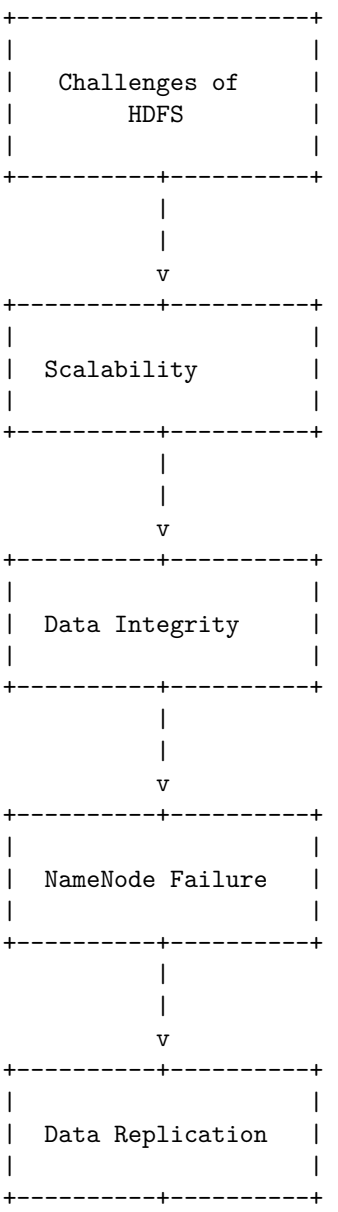
Benefits of HDFS

Fault Tolerance	Scalability	Data Locality
HDFS is designed to be highly fault-tolerant. It can automatically recover from hardware failures and continue to operate without significant interruption.	HDFS can easily scale to handle large amounts of data by adding more nodes to the cluster.	HDFS moves computation to the data, rather than moving data to the computation. This reduces network congestion and increases the overall throughput of the system.
Cost Effective	**High Throughput**	**Reliability**
HDFS is designed to run on commodity hardware, which makes it a cost-effective solution for storing large amounts of data.	HDFS is optimized for high throughput of large data sets. It can handle hundreds of megabytes to gigabytes of data per second.	HDFS provides reliable data storage by replicating data across multiple nodes. This ensures that data is available even if

```
|                      |                      | some nodes fail.     |
+----------------------+----------------------+----------------------+
```

Challenges of HDFS

Here is an ASCII diagram that illustrates some of the challenges of HDFS:

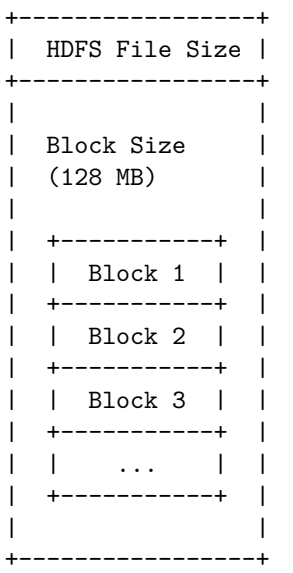


Some of the challenges of HDFS include scalability, data integrity, NameNode

failure, and data replication. Scalability refers to the ability of the system to handle increasing amounts of data and users. Data integrity refers to the accuracy and consistency of data stored in the system. NameNode failure refers to the potential for the single point of failure in the HDFS architecture. Data replication refers to the need to replicate data across multiple nodes to ensure data availability and durability.

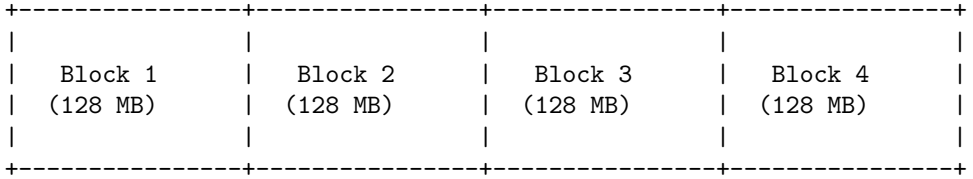
File sizes in HDFS

Here is an ASCII diagram that represents the file sizes in HDFS:



Block sizes in HDFS

Hadoop Distributed File System (HDFS) is a distributed file system designed to store large files across multiple machines. HDFS stores files as blocks, and the default block size is 128 MB. Here is an ASCII diagram that illustrates how a file is split into blocks in HDFS:



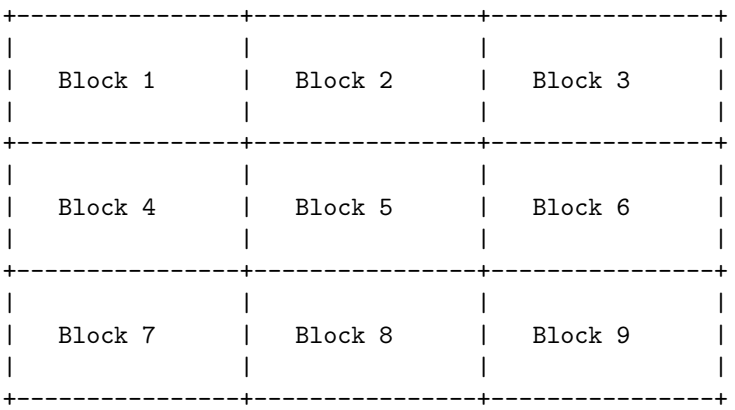
Each block is stored on a different DataNode in the HDFS cluster. The NameNode keeps track of the location of each block and coordinates access to the file data.

Block Abstraction in HDFS

HDFS is a distributed file system that stores large data sets across multiple machines. It is designed to scale up from a single server to thousands of machines, each providing local storage and computation. One of the key concepts in HDFS is the abstraction of a block.

In HDFS, files are split into blocks of a fixed size (by default, 128 MB) and these blocks are stored across multiple machines in the cluster. Each block is replicated multiple times (by default, 3 times) to ensure data availability and fault tolerance.

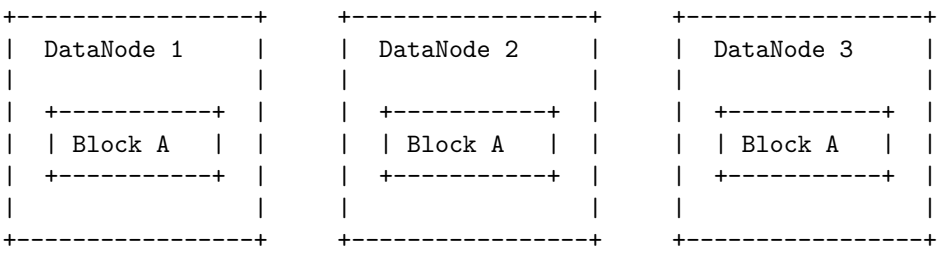
Here is an ASCII diagram that illustrates the block abstraction in HDFS:



In this diagram, each square represents a block of data. The blocks are distributed across multiple machines in the cluster, and each block is replicated multiple times to ensure data availability and fault tolerance.

Data replication in HDFS

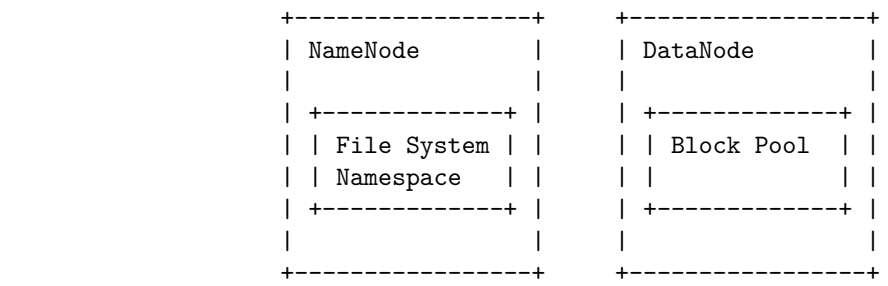
Here is an ASCII diagram that illustrates the data replication process in Hadoop Distributed File System (HDFS):



In HDFS, data is stored in blocks and replicated across multiple DataNodes for fault tolerance. In this example, Block A is replicated across DataNode

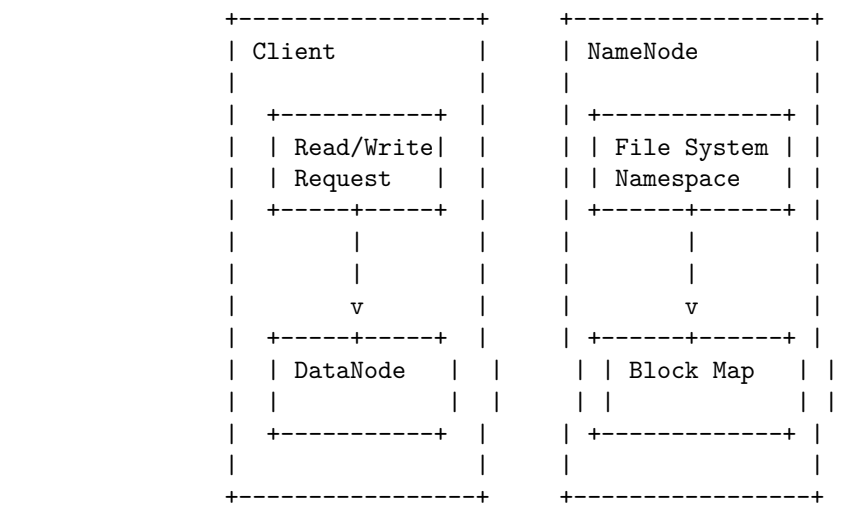
1, DataNode 2, and DataNode 3. This ensures that even if one or two of the DataNodes fail, the data is still available on the remaining DataNode(s).

Hadoop Distributed File System (HDFS) is a distributed file system designed to store large data sets across multiple machines. Here is an ASCII diagram that shows how HDFS stores data:



In HDFS, data is split into blocks and distributed across multiple DataNodes. The NameNode is responsible for managing the file system namespace and regulating access to files by clients. The DataNodes are responsible for storing the data blocks and serving read and write requests from the clients.

How does HDFS store



When a client wants to read or write data, it sends a request to the NameNode. The NameNode then returns the location of the data blocks to the client. The client then communicates directly with the DataNodes to read or write the data.

Read operations in HDFS

Here is an ASCII diagram that illustrates the read operations in HDFS:

```
Client
  |
  | 1. Open file
  v
NameNode
  |
  | 2. Return block locations
  v
Client
  |
  | 3. Read data from DataNodes
  v
DataNode(s)
```

1. The client opens a file in HDFS by calling the `open()` method on the `FileSystem` object, which sends a request to the NameNode.
2. The NameNode returns the block locations for the file to the client.
3. The client reads the data from the DataNodes that store the blocks of the file.

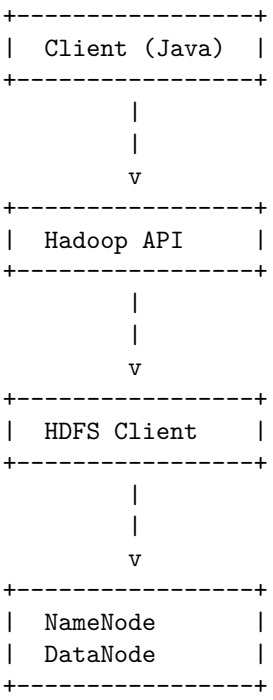
Write Operations in HDFS

Hadoop Distributed File System (HDFS) is a distributed file system designed to store large data sets reliably and to stream those data sets at high bandwidth to user applications. Here are some key points about write operations in HDFS:

1. **Data Replication:** HDFS replicates data blocks for fault tolerance. The default replication factor is 3, meaning that HDFS stores three copies of each data block.
2. **Data Pipelining:** When a client writes data to HDFS, the data is first written to the local disk of the client machine. Then, the data is sent to the first DataNode in the pipeline. The first DataNode stores the data and forwards it to the second DataNode in the pipeline, and so on.
3. **Data Integrity:** HDFS uses checksums to ensure data integrity. When a client writes data to HDFS, it computes a checksum for each data block and sends the checksum to the DataNode along with the data. The DataNode verifies the checksum before storing the data.
4. **Write-once-read-many:** HDFS is a write-once-read-many file system. Once a file is created, it cannot be modified. However, it can be appended to or overwritten.
5. **Atomicity:** HDFS supports atomic writes. When a client writes data to HDFS, the data is either written completely or not at all. If a write operation fails, the file system state remains unchanged.

Java interfaces to HDFS

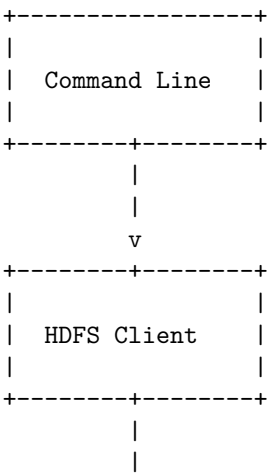
Here is an ASCII diagram that shows the Java interfaces to HDFS:

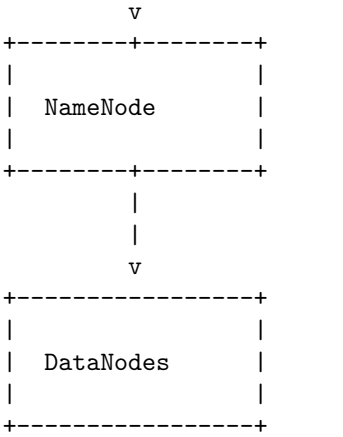


The diagram shows the flow of data from a Java client to the Hadoop Distributed File System (HDFS). The client interacts with the Hadoop API, which in turn communicates with the HDFS client. The HDFS client then communicates with the NameNode and DataNode to store and retrieve data from the HDFS.

Command Line Interface to HDFS

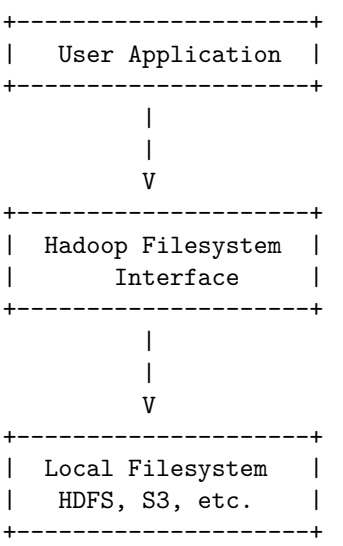
Here is an ASCII diagram that shows the command line interface to HDFS:





The command line interface allows users to interact with HDFS by entering commands. The HDFS client processes these commands and communicates with the NameNode to perform operations on the file system. The NameNode manages the file system namespace and regulates access to files by clients. The DataNodes store and retrieve data blocks as directed by the NameNode.

Hadoop file system interfaces



The Hadoop filesystem interface provides a common abstraction for different filesystem implementations, such as local filesystem, HDFS, S3, etc. This allows user applications to interact with different filesystems using the same interface. The diagram above illustrates the relationship between the user application, the Hadoop filesystem interface, and the underlying filesystem implementation.

Data flow in HDFS

Here is an ASCII diagram that illustrates the data flow in HDFS:

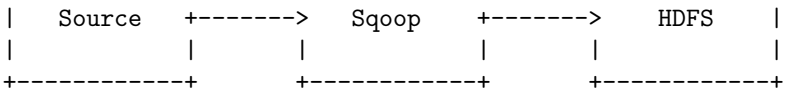
```
Client
   |
   | write request
   v
NameNode
   |
   | block locations
   v
Client
   |
   | write data
   v
DataNode 1
   |
   | replicate data
   v
DataNode 2
   |
   | replicate data
   v
DataNode 3
```

When a client wants to write data to HDFS, it sends a write request to the NameNode. The NameNode responds with the block locations where the data should be written. The client then writes the data to the first DataNode. The first DataNode replicates the data to the second DataNode, which in turn replicates the data to the third DataNode. This ensures that the data is stored redundantly across multiple DataNodes for fault tolerance.

Data Ingest with Flume and Sqoop in HDFS

Here is an ASCII diagram that illustrates the process of data ingest with Flume and Sqoop in HDFS:

```
+------------+       +------------+       +------------+
|            |       |            |       |            |
|   Source   +------->   Flume    +------->    HDFS    |
|            |       |            |       |            |
+------------+       +------------+       +------------+
       |                    |
       |                    |
       v                    v
+------------+       +------------+
|            |       |            |
```

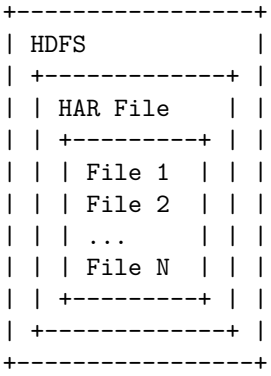


Flume and Sqoop are two tools used for data ingestion into Hadoop Distributed File System (HDFS). Flume is used for collecting, aggregating, and moving large amounts of streaming data into HDFS, while Sqoop is used for efficiently transferring bulk data between Hadoop and structured data stores such as relational databases.

In the diagram above, data from various sources is ingested into HDFS using either Flume or Sqoop. The sources send data to Flume, which then forwards it to HDFS. Alternatively, Sqoop can be used to import data from the sources directly into HDFS.

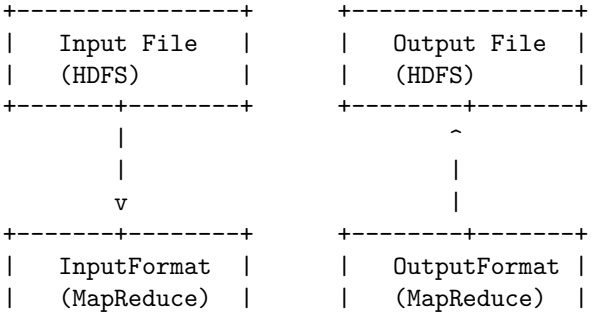
Hadoop archives in HDFS

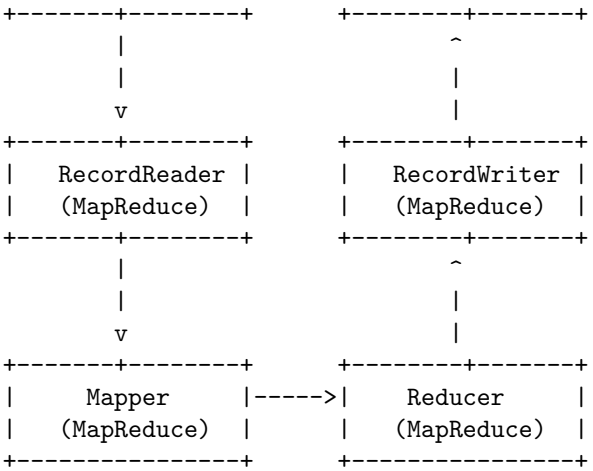
Hadoop archives (HAR) are a way to reduce the number of files in HDFS by combining small files into larger ones. Here is an ASCII diagram that shows how Hadoop archives work in HDFS:



Hadoop I/O

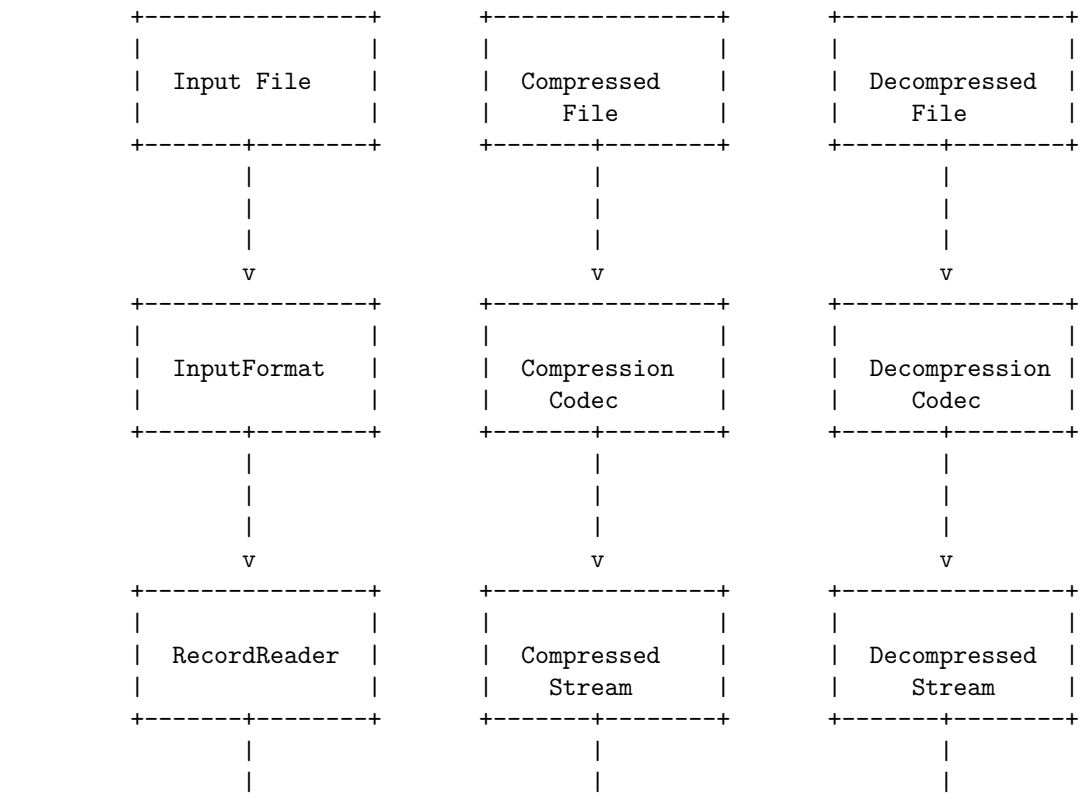
Here is an ASCII diagram for Hadoop I/O:

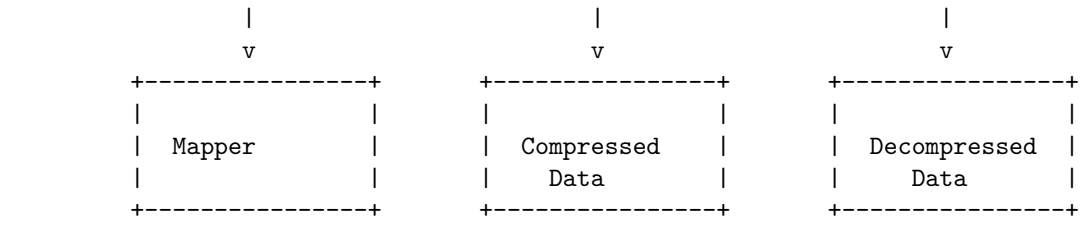




Compression in Hadoop IO

Here is an ASCII diagram that illustrates the process of compression in Hadoop IO:

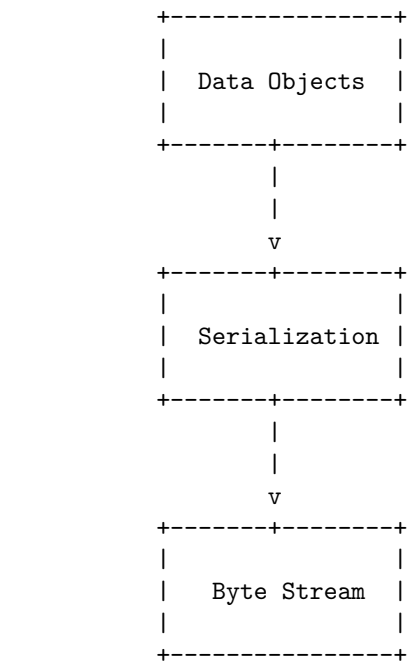




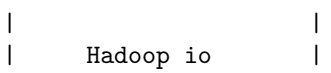
This diagram shows the flow of data from an input file, through the InputFormat and RecordReader, to the Mapper. The data can be compressed using a Compression Codec, which creates a compressed file and a compressed stream of data. The compressed data can then be decompressed using a Decompression Codec, which creates a decompressed file and a decompressed stream of data. The decompressed data can then be processed by the Mapper.

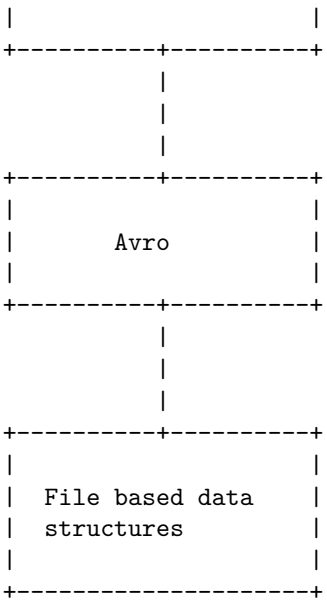
Serialization in Hadoop IO

Here is an ASCII diagram that illustrates the process of serialization in Hadoop IO:



Avro and file based data structures in Hadoop io





Hadoop Environment

Hadoop is an open-source software framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models. The Hadoop environment consists of several components, including:

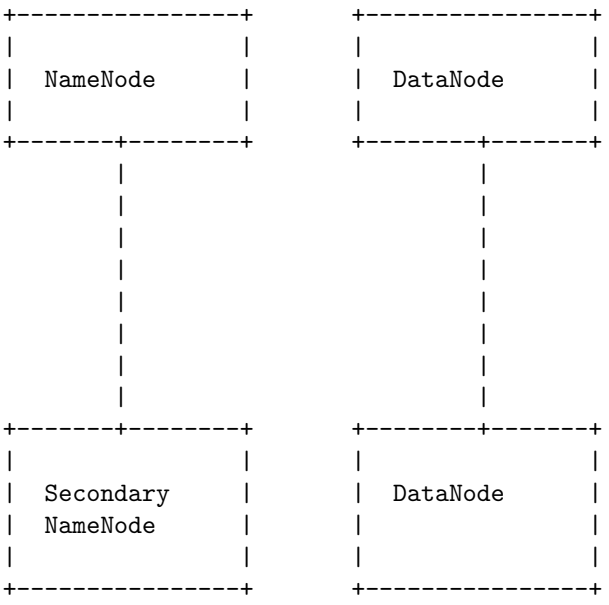
1. **Hadoop Distributed File System (HDFS)**: A distributed file system that provides high-throughput access to application data.
2. **MapReduce**: A programming model for processing large data sets with a parallel, distributed algorithm on a cluster.
3. **YARN**: A resource management platform responsible for managing compute resources in clusters and using them for scheduling of users' applications.
4. **Hadoop Common**: A set of common utilities that support the other Hadoop modules.

To set up a Hadoop environment, one needs to install and configure these components on a cluster of computers. This can be done manually or using tools such as Apache Ambari, which provides an easy-to-use web-based interface for provisioning, managing, and monitoring Hadoop clusters.

Once the Hadoop environment is set up, users can submit their data processing jobs to the cluster, which will be scheduled and executed by the YARN resource manager. The results of the processing can then be retrieved from the HDFS distributed file system.

Setting up a Hadoop cluster in Hadoop Environment

Here is an ASCII diagram that shows the process of setting up a Hadoop cluster in a Hadoop environment:



In a Hadoop environment, a cluster is set up with a NameNode, a Secondary NameNode, and multiple DataNodes. The NameNode is responsible for managing the file system namespace and regulating access to files by clients. The Secondary NameNode is responsible for performing periodic checkpoints of the namespace and helps keep the file system metadata in sync. The DataNodes are responsible for storing the actual data in HDFS.

Cluster Specification in Hadoop Environment

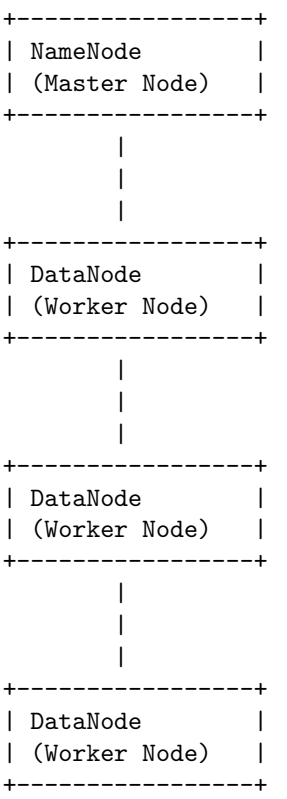
A Hadoop cluster is a special type of computational cluster designed specifically for storing and analyzing huge amounts of unstructured data in a distributed computing environment. Such clusters run Hadoop's open-source distributed processing software on low-cost commodity computers.

- A Hadoop cluster is designed to store and analyze large amounts of structured, semi-structured, and unstructured data in a distributed environment.
- It is often referred to as a shared-nothing system because the only thing that is shared between the nodes is the network itself.
- A Hadoop cluster is a collection of computers, known as nodes, that are networked together to perform parallel computations on big data sets.
- Unlike other computer clusters, Hadoop clusters are designed specifically to store and analyze mass amounts of structured and unstructured data

in a distributed computing environment.
- To configure the Hadoop cluster you will need to configure the environment in which the Hadoop daemons execute as well as the configuration parameters for the Hadoop daemons.
- The Hadoop daemons are NameNode / DataNode and JobTracker / TaskTracker.

Cluster Setup and Installation in Hadoop Environment

Here is an ASCII diagram that illustrates the cluster setup and installation in a Hadoop environment:



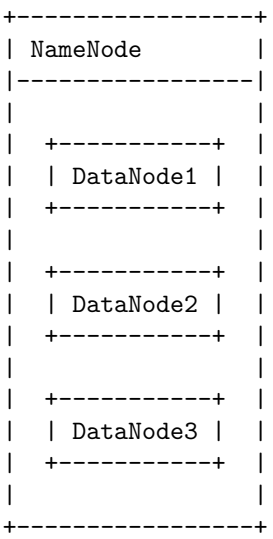
In a Hadoop cluster, there is one NameNode (Master Node) that manages the file system namespace and regulates access to files by clients. There are also multiple DataNodes (Worker Nodes) that store and retrieve data blocks and report to the NameNode. The NameNode and DataNodes communicate with each other using the Hadoop Distributed File System (HDFS) protocol.

To set up a Hadoop cluster, you need to install Hadoop on all the nodes (NameNode and DataNodes) and configure them properly. The installation process may vary depending on the operating system and the version of Hadoop you

are using. You can find detailed instructions on the Apache Hadoop website or in the Hadoop documentation.

Hadoop Configuration in Hadoop Environment

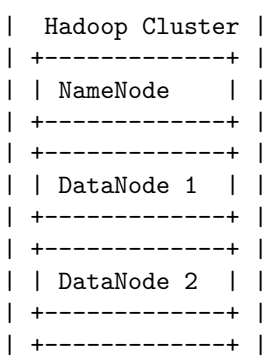
Here is an ASCII diagram that shows the Hadoop configuration in a Hadoop environment:

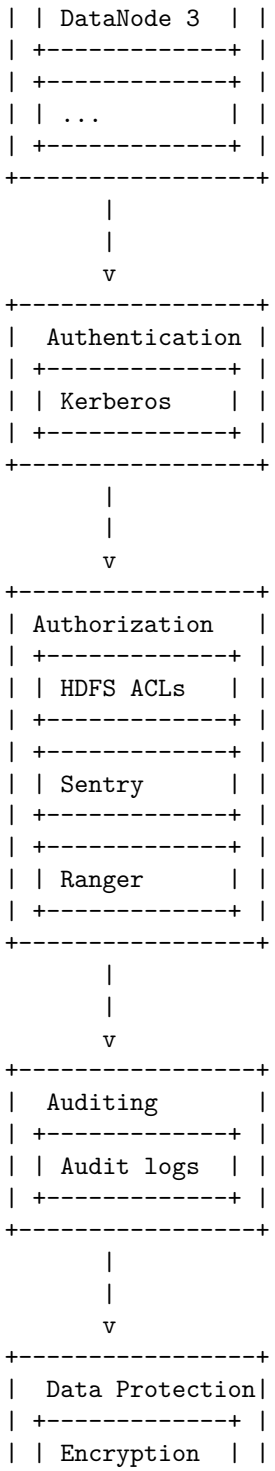


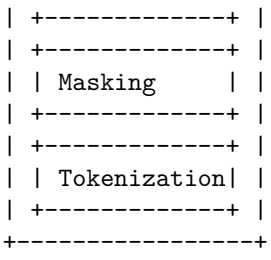
In this diagram, the NameNode is the master node that manages the file system namespace and regulates access to files by clients. The DataNodes are the worker nodes that store and retrieve data blocks. The NameNode communicates with the DataNodes to manage the storage and retrieval of data blocks.

Security in Hadoop in Hadoop Environment

Here is an ASCII diagram that shows the security features in a Hadoop environment:

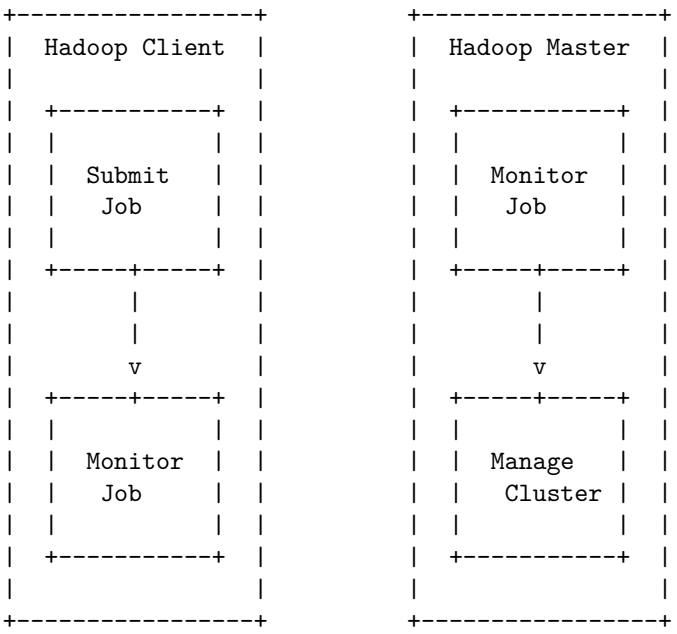



| DataNode 3 |
| ... |
Authentication
Kerberos
Authorization
HDFS ACLs
Sentry
Ranger
Auditing
Audit logs
Data Protection
Encryption



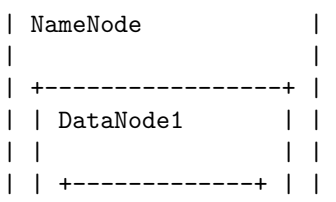
Administering Hadoop in Hadoop Environment

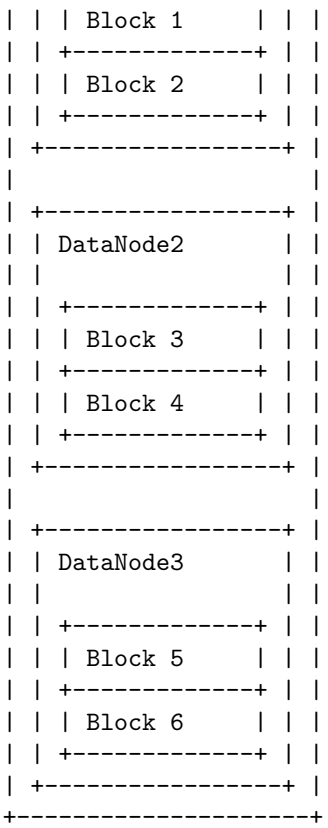
Here is an ASCII diagram that shows the process of administering Hadoop in a Hadoop environment:



HDFS monitoring & maintenance in Hadoop Environment

Here is an ASCII diagram that illustrates the HDFS monitoring and maintenance in a Hadoop environment:



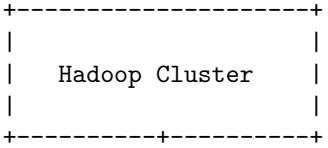


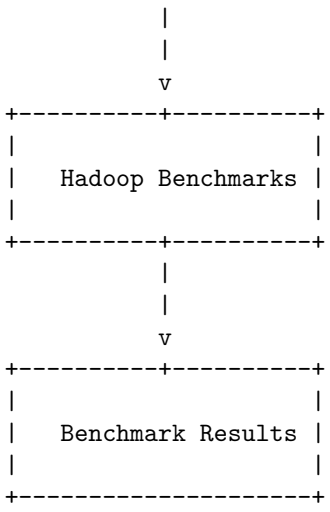
In this diagram, the NameNode is responsible for managing the file system namespace and regulating access to files by clients. The DataNodes are responsible for storing the data blocks and performing block creation, deletion, and replication upon instruction from the NameNode. The blocks represent the data stored in the HDFS.

Monitoring and maintenance of the HDFS involves keeping track of the health and status of the NameNode and DataNodes, as well as ensuring that data is properly replicated and balanced across the DataNodes. This can be done through tools such as the Hadoop web UI, JMX, and log analysis.

Hadoop benchmarks in Hadoop Environment

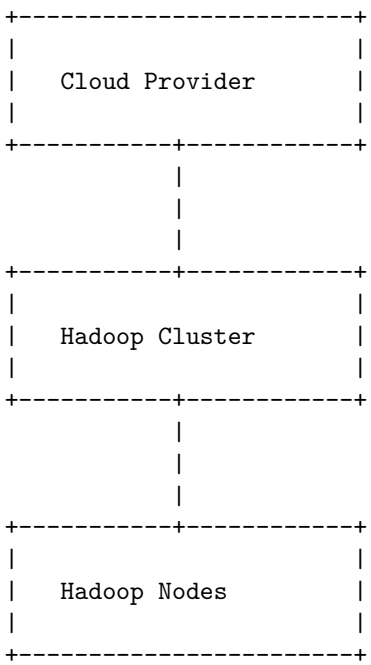
Here is an ASCII diagram of Hadoop benchmarks in a Hadoop environment:





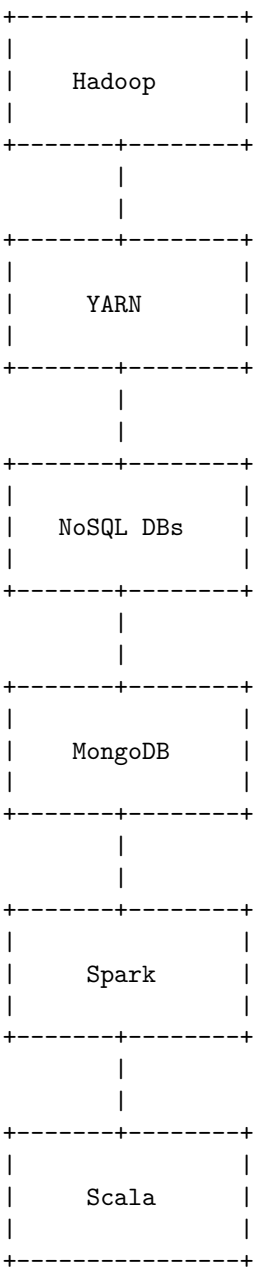
Hadoop in the cloud in Hadoop Environment

Here is an ASCII diagram that represents a Hadoop environment in the cloud:



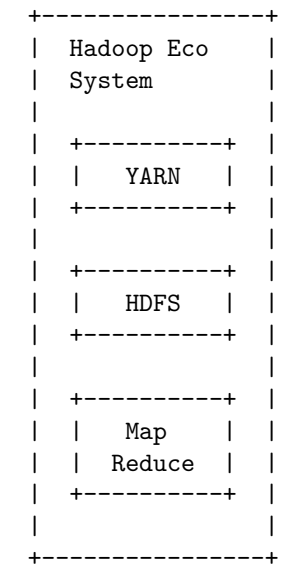
Unit 3 - Hadoop Eco System and YARN , no SQL databases , MongoDB , Spark , Scala

Here is an ASCII diagram that represents the Hadoop ecosystem and its components, including YARN, NoSQL databases, MongoDB, Spark, and Scala:



Hadoop Eco System and YARN

Here is an ASCII diagram of the Hadoop Eco System and YARN:



YARN (Yet Another Resource Negotiator) is the resource management layer of Hadoop. It is responsible for managing and allocating resources for the various applications running on the Hadoop cluster. HDFS (Hadoop Distributed File System) is the storage layer of Hadoop, responsible for storing and managing data across the cluster. MapReduce is a programming model for processing large data sets in parallel across the cluster.

Hadoop Ecosystem Components

The Apache Hadoop ecosystem refers to the various components of the Apache Hadoop software library. It includes open source projects as well as a complete range of complementary tools. The Hadoop ecosystem architecture is made up of four main components: data storage, data processing, data access, and data management.

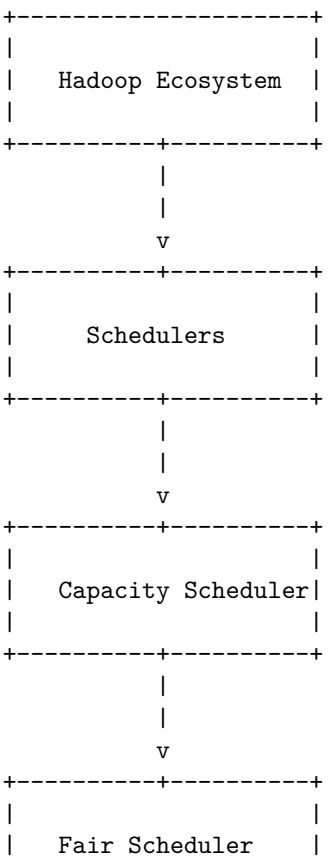
Some of the most well-known tools of the Hadoop ecosystem include:

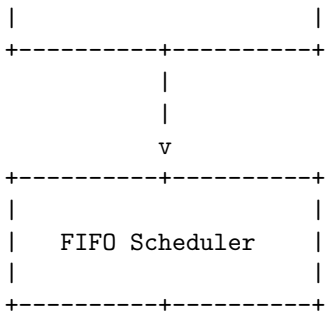
1. **HDFS (Hadoop Distributed File System)**: A distributed file system that provides high-throughput access to application data.
2. **Hive**: An open source data warehouse system for querying and analyzing large datasets stored in Hadoop files. Hive performs three main functions: data summarization, query, and analysis. Hive uses a language called HiveQL (HQL), which is similar to SQL.
3. **Pig**: A high-level platform for creating MapReduce programs used with Hadoop.

4. **YARN (Yet Another Resource Negotiator)**: A framework for job scheduling and cluster resource management.
5. **MapReduce**: A programming model for processing large data sets with a parallel, distributed algorithm on a cluster.
6. **Spark**: An open source, distributed computing system that can process large amounts of data quickly.
7. **HBase**: A distributed, column-oriented database built on top of HDFS.
8. **Oozie**: A workflow scheduler system to manage Apache Hadoop jobs.
9. **Sqoop**: A tool designed for efficiently transferring bulk data between Apache Hadoop and structured data stores such as relational databases.
10. **Zookeeper**: A centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services.

Each of these components plays a specific role in the Hadoop ecosystem and helps to make it a powerful tool for big data processing.

Schedulers in Hadoop Ecosystem





Fair and Capacity in Hadoop Ecosystem

Hadoop is a batch processing ecosystem that cannot analyze data on-the-fly. In Hadoop, there are mainly 3 types of Schedulers: FIFO (First In First Out) Scheduler, Capacity Scheduler, and Fair Scheduler. These Schedulers are actually a kind of algorithm that we use to schedule tasks in a Hadoop cluster when we receive requests from different clients.

Fair Scheduler allows YARN applications to justly share resources in large Hadoop clusters. With this scheduler, there is no need for reserving a set amount of capacity because it will dynamically balance resources between all running applications. Fair scheduling is a method of assigning resources to jobs such that all jobs get, on average, an equal share of resources over time. When there is a single job running, that job uses the entire cluster. When other jobs are submitted, tasks slots that free up are assigned to the new jobs, so that each job gets roughly the same amount of CPU time.

The two top tools to schedule a job in Hadoop are Capacity and Fair. The Fair Scheduler is very much similar to that of the capacity scheduler. The priority of the job is kept in consideration.

Hadoop 2.0 New Features - NameNode high availability

- Hadoop 2.0 introduced the High Availability feature to solve the Single Point of Failure (SPOF) problem in the older versions of Hadoop .
- Hadoop 2.0 overcomes this SPOF shortcoming by providing support for multiple NameNodes .
- It brings in an extra NameNode (Passive Standby NameNode) to the Hadoop Architecture which is configured for automatic failover .
- The main motive of the Hadoop 2.0 High Availability project is to render availability to large data applications 24/7 through the deployment of 2 Hadoop Name Nodes .
- This eliminates the NameNode as a potential single point of failure (SPOF) in an HDFS cluster .

HDFS Federation in Hadoop Ecosystem

HDFS Federation is a feature introduced in Hadoop 2 that enhances the existing HDFS architecture. It overcomes the limitations of the previous HDFS architecture by adding support for multiple NameNodes/namespaces to HDFS. This allows the use of more than one NameNode/namespace .

The HDFS Federation architecture has a collection of Namespace volumes, which are self-contained management units. When a NameNode or namespace is deleted, the corresponding block pool present in the DataNodes also gets deleted. When upgrading the cluster, each namespace volume is upgraded as a unit .

HDFS has two main layers: Namespace and Block Storage Service. The Namespace layer consists of directories, files, and blocks and supports all namespace-related file system operations such as creating, deleting, modifying, and listing files and directories. The Block Storage Service has two parts: Block Management (performed in the NameNode) and Block Storage (performed in the DataNodes) .

Overall, the HDFS Federation feature added to Hadoop 2.x provides support for multiple NameNodes/namespaces, overcoming the isolation, scalability, and performance limitations of the prior HDFS architecture. It also opens up the architecture for future innovations .

MRv2 in Hadoop ecosystem

- MRv2, also known as Hadoop 2, is a version of Hadoop where the resource management and scheduling tasks are separated from MapReduce by YARN (Yet Another Resource Negotiator). The resource management and scheduling layer lies beneath the MapReduce layer.
- MRv2 is an application framework that runs within YARN.
- In Hadoop version 1, MapReduce was responsible for both processing and cluster resource management. In Apache Hadoop version 2, cluster resource management has been moved from MapReduce into YARN, thus enabling other application engines to utilize YARN and Hadoop, while also improving the performance of MapReduce.
- Apache Hadoop MapReduce 2.x (MRv2) supports backward compatibility of org.apache.hadoop.mapred APIs. Binary compatibility here means that the compiled binaries should be able to run without any modification on the new framework.

YARN

Yarn is a long continuous length of interlocked fibers, used in sewing, crocheting, knitting, weaving, embroidery, ropemaking, and the production of textiles. Thread is a type of yarn intended for sewing by hand or machine.

Yarn comes in an amazing variety of types and styles, allowing you to personalize your project with color and texture by choosing a yarn that fits your style. If

you find a knit or crochet pattern you like, look at the yarn it recommends.

Running MRv1 in YARN

- MRv1, also known as MapReduce version 1, is a framework for processing large data sets in a distributed computing environment.
- YARN, or Yet Another Resource Negotiator, is a resource management layer in Hadoop that allows multiple data processing engines to share resources in a cluster.
- To run MRv1 in YARN, the following steps can be taken:
    1. Install and configure Hadoop with YARN.
    2. Set up the MapReduce job configuration with the appropriate settings for running in YARN.
    3. Submit the MapReduce job to the YARN resource manager for execution.
    4. Monitor the progress of the job and retrieve the results once it has completed.
- Running MRv1 in YARN allows for more efficient use of cluster resources and improved scalability compared to running MRv1 without YARN.

NoSQL Databases

NoSQL databases, also known as "non-SQL" or "non-relational" databases, provide a mechanism for storing and retrieving data that is modeled in means other than the tabular relations used in relational databases. The data structures used by NoSQL databases, such as key-value pairs, wide columns, graphs, or documents, are different from those used by default in relational databases, making some operations faster in NoSQL. The particular suitability of a given NoSQL database depends on the problem it must solve.

NoSQL databases are designed to be used across large distributed systems and are much more scalable and faster at handling very large data loads than traditional relational databases. Unlike other databases, NoSQL databases do not use the standard tabular relationships that relational databases employ.

NoSQL databases come in a variety of types based on their data model. The main types are document, key-value, wide-column, and graph. They provide flexible schemas and scale easily with large amounts of data and high user loads.

Some popular NoSQL databases include Apache CouchDB, Elasticsearch, and Couchbase.

Introduction to NoSQL databases

NoSQL databases are non-relational databases that store and retrieve data in ways that do not involve the use of a fixed schema like traditional relational databases. Some of the key characteristics of NoSQL databases include:

1. **Schema-less:** NoSQL databases do not require a fixed schema and can handle unstructured and semi-structured data.
2. **Scalability:** NoSQL databases are designed to scale horizontally, meaning that they can handle large amounts of data by distributing it across multiple servers.
3. **Flexibility:** NoSQL databases allow for flexible data modeling, making it easier to make changes to the data structure without having to make changes to the entire database.
4. **Performance:** NoSQL databases are optimized for specific data access patterns, which can result in faster performance for certain types of queries.

Some common types of NoSQL databases include document databases, key-value stores, column-family stores, and graph databases. Each type of NoSQL database is designed to handle specific data access patterns and use cases.

NoSQL databases are commonly used in big data and real-time web applications, where the ability to handle large amounts of unstructured data and scale horizontally is important. Some popular NoSQL databases include MongoDB, Cassandra, Redis, and Neo4j.

MongoDB

MongoDB is a cross-platform document-oriented database program. It is classified as a NoSQL database program and uses JSON-like documents with optional schemas. Some key features of MongoDB include:

1. **Flexible data model:** MongoDB stores data in flexible, JSON-like documents, meaning fields can vary from document to document and data structure can be changed over time.
2. **Scalability:** MongoDB is horizontally scalable, allowing for easy addition of more machines to support data growth.
3. **High performance:** MongoDB provides high performance for both reads and writes, and includes support for in-place updates and flexible indexing.
4. **Expressive query language:** MongoDB provides a rich query language that allows for filtering and sorting by any field, as well as aggregations and geospatial queries.
5. **Strong consistency:** MongoDB provides strong consistency, meaning that reads and writes are always made to an up-to-date version of the data.

Overall, MongoDB is a powerful and flexible database program that is widely used in a variety of applications. It is particularly well-suited for handling large amounts of unstructured or semi-structured data.

Introduction to MongoDB

MongoDB is a cross-platform document-oriented database program. It is classified as a NoSQL database program, which means it does not use the traditional tabular relational database structure. Instead, it uses JSON-like documents with optional schemas.

Some key features of MongoDB include:

- **Document-based**: Data is stored in flexible, JSON-like documents, meaning fields can vary from document to document and data structure can be changed over time.
- **Ad hoc queries**: MongoDB supports field, range, and regular expression queries, and can also search within documents and arrays.
- **Indexing**: Fields in a MongoDB document can be indexed with primary and secondary indices.
- **Aggregation**: MongoDB provides an aggregation framework for data analysis and transformation.
- **Replication**: MongoDB provides high availability with replica sets, which consist of two or more copies of the data.
- **Sharding**: MongoDB supports horizontal scaling through sharding, which distributes data across multiple machines.
- **File storage**: MongoDB can be used as a file system, taking advantage of its load balancing and data replication features.
- **Server-side JavaScript execution**: JavaScript can be used in queries, aggregation functions, and sent directly to the database to be executed.

MongoDB is widely used for its flexibility, scalability, and performance. It is commonly used for web and mobile applications, real-time analytics, and content management systems. It is developed by MongoDB Inc. and is published under the Server Side Public License (SSPL).

Data Types in MongoDB

MongoDB supports several data types, including:

1. **String**: This is the most commonly used data type to store data. Strings in MongoDB must be UTF-8 valid.
2. **Integer**: This type is used to store a numerical value. Integer can be 32-bit or 64-bit, depending on the server architecture.
3. **Boolean**: This type is used to store a Boolean (true/ false) value.
4. **Double**: This type is used to store floating-point values.
5. **Min/ Max keys**: This type is used to compare a value against the lowest and highest BSON elements, respectively.
6. **Arrays**: This type is used to store arrays or lists.

7. **Object**: This type is used to store embedded documents.
8. **Null**: This type is used to store a Null value.
9. **Symbol**: This type is used identically to a string; however, it's generally reserved for languages that use a specific symbol type.
10. **Date**: This type is used to store the current date or time in UNIX time format. You can specify your own date time by creating an object of Date and passing the desired date time string as a parameter.
11. **Object ID**: This type is used to store the document's ID.
12. **Binary data**: This type is used to store binary data.
13. **Code**: This type is used to store JavaScript code into the document.
14. **Regular expression**: This type is used to store regular expression.

These are the most commonly used data types in MongoDB. Each data type has its own specific use and can be used to store different types of data in a MongoDB database.

Creating Documents in MongoDB

MongoDB is a document-based database that stores data in flexible, JSON-like documents. Here are the steps to create a document in MongoDB:

1. **Connect to the MongoDB server**: Use the `mongo` shell or a MongoDB driver to connect to the MongoDB server.
2. **Select a database**: Use the `use` command to select the database where you want to create the document.
3. **Create a collection**: If the collection where you want to store the document does not exist, create it using the `db.createCollection()` method.
4. **Insert the document**: Use the `db.collection.insertOne()` or `db.collection.insertMany()` method to insert one or many documents into the collection.

Here is an example of creating a document in MongoDB using the `mongo` shell:

```
// Connect to the MongoDB server
mongo

// Select the database
use myDatabase

// Create a collection
db.createCollection('myCollection')

// Insert a document
db.myCollection.insertOne({name: 'John', age: 25})
```

This will create a new document in the `myCollection` collection of the `myDatabase` database, with the fields `name` and `age` set to `'John'` and `25`,

respectively.

Updating Documents in MongoDB

MongoDB provides several methods to update documents within a collection. Here are some key points to remember when updating documents in MongoDB:

1. The `updateOne()` method updates a single document that matches the specified filter.
2. The `updateMany()` method updates all documents that match the specified filter.
3. The `$set` operator is used to update specific fields within a document.
4. The `$inc` operator is used to increment the value of a field by a specified amount.
5. The `$push` operator is used to add an element to an array field.
6. The `$pull` operator is used to remove an element from an array field.
7. The `upsert` option can be used to insert a new document if no document matches the specified filter.
8. The `multi` option can be used to update multiple documents that match the specified filter.

These are some of the key points to remember when updating documents in MongoDB. It is important to carefully consider the update operation and the potential impact on the data before performing an update.

Deleting Documents in MongoDB

MongoDB provides several methods to delete documents from a collection:

1. `deleteOne()`: This method deletes a single document that matches the specified filter.
2. `deleteMany()`: This method deletes all documents that match the specified filter.
3. `findOneAndDelete()`: This method finds a single document that matches the specified filter and deletes it, returning the deleted document.

Here is an example of how to use the `deleteOne()` method to delete a document from a collection:

```
db.collection.deleteOne({ name: "John" });
```

This command will delete the first document in the collection where the `name` field is equal to "John".

Similarly, here is an example of how to use the `deleteMany()` method to delete multiple documents from a collection:

```
db.collection.deleteMany({ age: { $lt: 18 } });
```

This command will delete all documents in the collection where the `age` field is less than 18.

Finally, here is an example of how to use the `findOneAndDelete()` method to find and delete a document from a collection:

```
db.collection.findOneAndDelete({ name: "Jane" });
```

This command will find the first document in the collection where the `name` field is equal to "Jane" and delete it, returning the deleted document.

It is important to note that when deleting documents from a collection, any indexes associated with the deleted documents will also be removed. Additionally, if the collection is capped, the `deleteOne()` and `deleteMany()` methods will not work and an error will be returned. In this case, the `findOneAndDelete()` method can be used instead.

Querying Documents in MongoDB

MongoDB is a document-based database that allows you to store and retrieve data in a flexible and scalable manner. One of the key features of MongoDB is its powerful query language, which allows you to retrieve documents from the database based on specific criteria.

Here are some key points to keep in mind when querying documents in MongoDB:

1. **Basic Queries**: To query documents in MongoDB, you can use the `find()` method on a collection. This method takes a query document as an argument, which specifies the criteria for the documents you want to retrieve.
2. **Query Operators**: MongoDB provides a wide range of query operators that you can use to build complex queries. These operators include comparison operators (e.g., `$eq`, `$gt`, `$lt`), logical operators (e.g., `$and`, `$or`, `$not`), and array operators (e.g., `$in`, `$all`).
3. **Projection**: When querying documents in MongoDB, you can use projection to specify which fields you want to include or exclude from the result set. This can be useful if you only need a subset of the data stored in the documents.
4. **Sorting**: You can use the `sort()` method to specify the order in which you want the documents to be returned. This method takes a sort document as an argument, which specifies the fields to sort on and the sort order (ascending or descending).
5. **Limiting and Skipping**: You can use the `limit()` and `skip()` methods to control the number of documents returned by a query and to skip over a certain number of documents. These methods can be useful for implementing pagination.
6. **Aggregation**: MongoDB provides a powerful aggregation framework that allows you to perform complex data processing and analysis on the server

side. You can use the `aggregate()` method to build aggregation pipelines that transform and analyze your data.

These are some of the key points to keep in mind when querying documents in MongoDB. By mastering the query language and understanding how to use the various query methods and operators, you can retrieve data from your MongoDB database in a flexible and efficient manner.

Indexing in MongoDB

- MongoDB uses indexing to make query processing more efficient. If there is no indexing, then MongoDB must scan every document in the collection and retrieve only those documents that match the query.
- Indexes are special data structures that store information related to the documents, making it easier for MongoDB to find the right data file. The indexes are ordered by the value of the field specified in the index.
- MongoDB provides a method called `createIndex()` that allows users to create an index.
- MongoDB has its ways of implementing indexing and offers various types. In MongoDB, we use the `createIndex` function to create an index and the `dropIndex` function to drop the index.
- MongoDB uses multikey indexes to index the content stored in arrays. If you index a field that holds an array value, MongoDB creates separate index entries for every element of the array. These multikey indexes allow queries to select documents that contain arrays by matching on element or elements of the arrays.
- MongoDB supports the creation of user-defined indexes on multiple fields, i.e. compound indexes. The order of the fields listed in a compound index is important.

Aggregation in MongoDB

Aggregation in MongoDB is the process of grouping data from multiple documents and performing operations on the grouped data to return a single result. It is similar to the GROUP BY clause in SQL.

Some key points to remember about aggregation in MongoDB are:

1. Aggregation operations can be performed on a collection using the `aggregate()` method.
2. The `aggregate()` method takes an array of aggregation pipeline stages as its argument.
3. Each stage in the pipeline processes the data and passes the result to the next stage.
4. Common pipeline stages include `$match`, `$group`, `$sort`, `$project`, and `$unwind`.
5. The `$group` stage is used to group documents by a specified expression and perform aggregation operations on the grouped data.

6. The `$project` stage is used to reshape the documents in the pipeline by including, excluding, or renaming fields.
7. The `$unwind` stage is used to deconstruct an array field from the input documents and output a document for each element in the array.
8. The result of the aggregation pipeline is a single document or an array of documents.

Capped Collections in MongoDB

- Capped collections are fixed-size collections that support high-throughput operations that insert and retrieve documents based on insertion order.
- Capped collections work in a way similar to circular buffers: once a collection fills its allocated space, it makes room for new documents by overwriting the oldest documents in the collection.
- You must create capped collections explicitly using the `db.createCollection()` method, which is a `mongosh` helper for the `create` command.
- When creating a capped collection you must specify the maximum size of the collection in bytes, which MongoDB will pre-allocate for the collection.
- Capped collection in MongoDB is basically used to store log information, the high volume of data, and cache information.
- Circular capped collection states that when we allocate the fixed size to the collection, it was exhausted. At that time capped collection in MongoDB will delete the oldest document from the collection.

Spark

Apache Spark is an open-source distributed general-purpose cluster-computing framework. It provides an interface for programming entire clusters with implicit data parallelism and fault tolerance. Some of the key features of Spark include:

1. **Speed:** Spark is designed to be fast, both for batch processing and for iterative algorithms. It can run programs up to 100 times faster than Hadoop MapReduce in memory, or 10 times faster on disk.
2. **Ease of Use:** Spark has easy-to-use APIs for operating on large datasets. It supports multiple languages including Python, Scala, and R.
3. **Generality:** Spark combines SQL, streaming, and complex analytics in a single engine. This makes it easy to combine different processing types and run them seamlessly in a single application.
4. **Runs Everywhere:** Spark runs on Hadoop, Mesos, standalone, or in the cloud. It can access diverse data sources including HDFS, Cassandra, HBase, and S3.

Spark has a number of built-in libraries, including support for SQL and DataFrames, MLlib for machine learning, GraphX for graph processing, and Streaming for stream processing. These libraries make it easy to build complex data processing pipelines and perform advanced analytics on large datasets.

Installing Spark

1. **Download Spark**: You can download the latest version of Spark from the Apache Spark website. Choose the package type that is suitable for your system and download it.
2. **Install Java**: Spark requires Java to be installed on your system. You can download and install the latest version of Java from the official website.
3. **Unpack Spark**: After downloading Spark, unpack the compressed file to a directory of your choice.
4. **Set Environment Variables**: Set the environment variables `SPARK_HOME` to the directory where you unpacked Spark and add `$SPARK_HOME/bin` to your `PATH` variable.
5. **Start Spark**: You can start Spark by running the `./bin/spark-shell` command from the Spark directory. This will start the Spark shell, where you can interactively run Spark commands.
6. **Test Spark**: To test if Spark is installed correctly, you can run a simple command in the Spark shell, such as `sc.parallelize(1 to 10).count()`, which should return the value `10`.
7. **Configure Spark**: You can configure Spark by editing the `conf/spark-defaults.conf` file in the Spark directory. This file contains default configuration options for Spark, which you can modify to suit your needs.

Spark Applications

- Spark Applications consist of a driver process and a set of executor processes.
- The driver process runs your main () function, sits on a node in the cluster, and is responsible for three things: maintaining information about the Spark Application; responding to a user's program or input; and analyzing, distributing, and scheduling work across the executors.
- Spark applications run as independent sets of processes on a cluster, coordinated by the driver program.
- The driver consists of your program, like a C# console app, and a Spark session. The Spark session takes your program and divides it into smaller tasks that are handled by the executors.
- A Spark application runs as independent processes, coordinated by the SparkSession object in the driver program. The resource or cluster manager assigns tasks to workers, one task per partition. A task applies its unit of work to the dataset in its partition and outputs a new partition dataset.
- For an in-depth overview of the API, start with the RDD programming guide and the SQL programming guide, or see "Programming Guides"

menu for other components. For running applications on a cluster, head to the deployment overview.
- Spark applications run as independent sets of processes on a pool, coordinated by the SparkContext object in your main program, called the driver program. The SparkContext can connect to the cluster manager, which allocates resources across applications. The cluster manager is Apache Hadoop YARN.

Jobs in Spark

- In Apache Spark, a job is a unit of work that is distributed across the cluster for parallel processing.
- A job is triggered when an action is called on a RDD or DataFrame/Dataset, such as `collect`, `count`, or `save`.
- A job is divided into stages, which are further divided into tasks. Each task is a unit of work that is executed on a single executor in the cluster.
- The number of stages in a job depends on the number of shuffle operations required to compute the result.
- The Spark scheduler is responsible for scheduling and managing the execution of jobs and tasks.
- Jobs can be monitored and managed using the Spark web UI or through the SparkContext API.
- Jobs can be submitted to a Spark cluster using the `spark-submit` command or through the Spark REST API.
- Jobs can be run in different modes, such as client mode or cluster mode, depending on the deployment configuration.
- Jobs can be configured with various settings, such as the number of executors, the amount of memory per executor, and the number of cores per executor.
- Jobs can be cancelled or killed using the Spark web UI or through the SparkContext API.

Stages and Tasks in Spark

Apache Spark is a distributed computing system that processes large datasets in parallel. The processing of data in Spark is divided into stages, and each stage is further divided into tasks.

1. **Stages:** A stage is a collection of tasks that can be executed in parallel. Stages are created based on the transformations in the Spark application. Transformations that have narrow dependencies, such as `map` and `filter`, can be grouped into a single stage. Transformations that have wide dependencies, such as `reduceByKey` and `join`, result in the creation of a new stage.

2. **Tasks:** A task is the smallest unit of work in Spark. Each task processes a partition of the data. The number of tasks in a stage is equal to the

number of partitions of the input data.

3. **Shuffling:** Shuffling is the process of redistributing data between stages. It occurs when data needs to be grouped by key or when data from multiple partitions needs to be combined. Shuffling can be an expensive operation, as it involves data movement across the network.
4. **DAGScheduler:** The DAGScheduler is responsible for dividing the Spark application into stages and creating tasks for each stage. It also determines the preferred location for each task, based on data locality.
5. **TaskScheduler:** The TaskScheduler is responsible for assigning tasks to executors for execution. It takes into account data locality and resource availability when making scheduling decisions.
6. **Execution:** Once the tasks are assigned to executors, they are executed in parallel. Each task reads its input data, performs the necessary transformations, and writes its output data. The output data of the final stage is returned to the driver program.

In summary, the processing of data in Spark is divided into stages, and each stage is further divided into tasks. The DAGScheduler is responsible for creating stages and tasks, while the TaskScheduler is responsible for assigning tasks to executors for execution. Shuffling is the process of redistributing data between stages, and it can be an expensive operation. The processing of data in Spark is highly parallel, with tasks being executed concurrently on multiple executors.

Resilient Distributed Databases in Spark

Resilient Distributed Datasets (RDD) is a fundamental data structure of Spark. It is an immutable distributed collection of objects. Each dataset in RDD is divided into logical partitions, which may be computed on different nodes of the cluster. RDDs can contain any type of Python, Java, or Scala objects, including user-defined classes .

RDDs are reliable and memory-efficient when it comes to parallel processing. By storing and processing data in RDDs, Spark speeds up MapReduce processes .

At the core, an RDD is an immutable distributed collection of elements of your data, partitioned across nodes in your cluster that can be operated in parallel with a low-level API that offers transformations and actions .

Spark can create distributed datasets from any storage source supported by Hadoop, including your local file system, HDFS, Cassandra, HBase, Amazon S3, etc. Spark supports text files, SequenceFiles, and any other Hadoop InputFormat. Text file RDDs can be created using SparkContext's textFile method .

Anatomy of a Spark Job Run

1. **Client Mode**: In client mode, the driver program runs on the client machine, which submits the Spark job.
2. **Cluster Mode**: In cluster mode, the driver program runs on a worker node within the cluster, and the client machine is only used to launch the job.
3. **Spark Context**: The first step in running a Spark job is to create a SparkContext object, which tells Spark how to access the cluster.
4. **Job Submission**: Once the SparkContext is created, the user can submit a Spark job by calling an action on an RDD or DataFrame.
5. **Stage Creation**: The Spark scheduler divides the job into stages, where each stage contains a sequence of transformations that can be executed in parallel.
6. **Task Scheduling**: Within each stage, the scheduler creates tasks, where each task processes a partition of the data.
7. **Task Execution**: The tasks are sent to the worker nodes for execution. Each task is executed in a separate thread within a JVM on the worker node.
8. **Shuffling**: If a stage requires data from multiple partitions of the previous stage, a shuffle operation is performed to redistribute the data.
9. **Result Collection**: Once all the tasks have completed, the results are collected and returned to the driver program.

Spark on YARN

- Apache Spark is a fast and general-purpose cluster computing system.
- It provides high-level APIs in Java, Scala, Python, and R, and an optimized engine that supports general computation graphs for data analysis.
- YARN (Yet Another Resource Negotiator) is the resource management layer of Hadoop.
- Spark can run on YARN, allowing it to take advantage of the resource management capabilities of YARN.
- When running Spark on YARN, each Spark executor runs as a YARN container.
- YARN allocates resources (CPU, memory, etc.) to the Spark application based on the resource requests specified in the Spark configuration.
- Spark on YARN can be run in two modes: client mode and cluster mode.
- In client mode, the Spark driver runs on the client machine, and the application master is only used for requesting resources from YARN.
- In cluster mode, the Spark driver runs inside an application master process, which is managed by YARN on the cluster.
- Running Spark on YARN allows for dynamic allocation of cluster resources, improving the efficiency of resource utilization.

SCALA

Scala is a general-purpose programming language that combines the features of object-oriented and functional programming. It was designed to be concise, expressive, and scalable. Some of the key features of Scala include:

1. **Object-Oriented:** Scala is a pure object-oriented language, meaning that every value is an object and every operation is a method call.
2. **Functional:** Scala also incorporates functional programming concepts, such as immutability, higher-order functions, and pattern matching.
3. **Type Inference:** Scala has a powerful type inference system, which allows the programmer to omit type annotations in many cases.
4. **Concurrency and Distribution:** Scala has built-in support for concurrency and distribution, making it easy to write programs that can run on multiple processors or machines.
5. **Interoperability:** Scala is designed to be interoperable with Java, meaning that Scala code can call Java libraries and vice versa.
6. **Scalability:** Scala is designed to be scalable, meaning that it can be used to write small scripts as well as large, complex systems.

Scala is used by many companies, including Twitter, LinkedIn, and Netflix, and is taught in many universities around the world. It is a popular choice for developing web applications, data processing pipelines, and distributed systems.

Introduction to Scala

Scala is a modern, multi-paradigm programming language designed to express common programming patterns in a concise, elegant, and type-safe way. It smoothly integrates features of object-oriented and functional languages.

Some key features of Scala include: - Scala is a statically typed language, which means that the type of a variable is checked at compile-time. - Scala is both object-oriented and functional. Every value is an object and every operation is a method call. - Scala has a concise syntax, which allows for the creation of complex data structures and operations with fewer lines of code. - Scala has a powerful type inference system, which allows the programmer to omit the type of a variable in many cases. - Scala has a rich standard library, which includes a wide range of data structures and operations. - Scala has built-in support for concurrency and parallelism, making it easier to write programs that can take advantage of multiple cores and processors.

Scala is a popular language for many applications, including web development, data science, and big data processing. It is used by many companies, including Twitter, LinkedIn, and Netflix. It is also a popular language for academic research in computer science. Scala is a language that is worth learning for

any programmer who wants to stay up-to-date with the latest developments in the field.

Classes and Objects in Scala

Scala is an object-oriented programming language, which means that it is based on the concept of objects. An object is an instance of a class, and a class is a blueprint for creating objects.

Here are some key points to remember about classes and objects in Scala:

1. A class is defined using the `class` keyword, followed by the name of the class and a body enclosed in curly braces.
2. The body of a class can contain fields and methods. Fields are variables that store the state of an object, while methods are functions that define the behavior of an object.
3. An object is created by calling the constructor of a class, which is a special method that is automatically called when an object is created.
4. The `new` keyword is used to create an object by calling the constructor of a class.
5. Objects can interact with each other by calling each other's methods.
6. Scala also supports the concept of companion objects, which are objects that share the same name as a class and are defined in the same source file. Companion objects can access private members of their corresponding class.

Here is an example of a simple class and object in Scala:

```
class Person(val name: String, val age: Int) {
  def greet(): Unit = {
    println(s"Hello, my name is $name and I am $age years old.")
  }
}

val p = new Person("John", 30)
p.greet() // prints "Hello, my name is John and I am 30 years old."
```

In this example, we define a `Person` class with two fields (`name` and `age`) and a `greet` method. We then create an object of the `Person` class using the `new` keyword and call its `greet` method. This prints a greeting message to the console.

Basic Types and Operators in Scala

Scala has a rich set of built-in data types and operators. Here are some of the basic types and operators in Scala:

1. **Numeric Types**: Scala has several numeric types, including `Byte`, `Short`, `Int`, `Long`, `Float`, and `Double`. These types represent 8-bit, 16-bit, 32-

bit, and 64-bit signed integers, as well as 32-bit and 64-bit floating-point numbers, respectively.

2. **Boolean Type**: The `Boolean` type in Scala has two values: `true` and `false`.
3. **Character Type**: The `Char` type in Scala represents a 16-bit Unicode character.
4. **String Type**: The `String` type in Scala represents a sequence of characters.
5. **Arithmetic Operators**: Scala supports the standard arithmetic operators, including `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), and `%` (modulus).
6. **Relational Operators**: Scala also supports the standard relational operators, including `==` (equal to), `!=` (not equal to), `>` (greater than), `<` (less than), `>=` (greater than or equal to), and `<=` (less than or equal to).
7. **Logical Operators**: The logical operators in Scala include `&&` (logical AND), `||` (logical OR), and `!` (logical NOT).
8. **Bitwise Operators**: Scala supports bitwise operators, including `&` (bitwise AND), `|` (bitwise OR), `^` (bitwise XOR), `~` (bitwise NOT), `<<` (left shift), `>>` (right shift), and `>>>` (unsigned right shift).

Built-in Control Structures in Scala

Scala has several built-in control structures that allow you to control the flow of your program. These include:

1. **If-else statements:** This is a conditional statement that allows you to execute a block of code if a certain condition is met, and another block of code if the condition is not met.
2. **While loops:** This is a loop that continues to execute a block of code as long as a certain condition is true.
3. **For loops:** This is a loop that iterates over a range of values or a collection of elements and executes a block of code for each iteration.
4. **Match expressions:** This is a powerful control structure that allows you to match a value against a set of patterns and execute a block of code based on the pattern that matches.
5. **Try-catch-finally expressions:** This is a control structure that allows you to handle exceptions in your code. You can use a try block to enclose code that might throw an exception, a catch block to handle the exception, and a finally block to execute code regardless of whether an exception was thrown or not.

These are some of the built-in control structures in Scala that you can use to control the flow of your program. They are similar to control structures in other programming languages, but with some differences in syntax and usage. It is important to understand these control structures and how to use them effectively in your Scala programs.

Functions and Closures in Scala

Scala is a functional programming language, which means that functions are first-class values. This means that functions can be assigned to variables, passed as arguments to other functions, and returned as values from other functions.

A function in Scala is defined using the `def` keyword, followed by the function name, parameters, and the function body. The return type of the function can be specified after the parameters, separated by a colon.

Here is an example of a simple function in Scala that takes two integers as arguments and returns their sum:

```
def add(x: Int, y: Int): Int = {
  x + y
}
```

Closures are functions that can access variables from their enclosing scope. This means that a closure can use variables that are not defined within the function itself, but are available in the surrounding context.

Here is an example of a closure in Scala:

```
val x = 10
val addX = (y: Int) => x + y
```

In this example, the `addX` function is a closure because it uses the `x` variable, which is defined outside of the function.

Closures are useful because they allow you to create functions that can operate on data that is not passed as arguments to the function. This can make your code more concise and easier to read.

In summary, functions and closures are important concepts in Scala and functional programming in general. Functions are first-class values that can be assigned to variables, passed as arguments, and returned as values. Closures are functions that can access variables from their enclosing scope, allowing them to operate on data that is not passed as arguments. These concepts allow for powerful and flexible programming techniques.

Inheritance in Scala

Inheritance is a fundamental concept in object-oriented programming that allows the creation of hierarchical classifications. In Scala, inheritance works in a similar way to other object-oriented languages such as Java.

- In Scala, a class can inherit from another class using the `extends` keyword.
- The class that is being inherited from is called the superclass, and the class that is inheriting from the superclass is called the subclass.
- The subclass inherits all the members (fields and methods) of the superclass, and can also add new members or override existing ones.
- In Scala, all classes inherit from a common superclass called `Any`, which provides basic methods such as `equals`, `hashCode`, and `toString`.
- Scala also supports multiple inheritance through the use of traits. A class can inherit from multiple traits using the `with` keyword.
- Traits are similar to interfaces in Java, but can also contain concrete methods and fields.
- Inheritance can be used to achieve code reuse and polymorphism, which allows objects of different classes to be treated as objects of a common superclass.

Hadoop Eco System Frameworks, Pig, Hive and HBase

The Hadoop Ecosystem is a framework and suite of tools that tackle the many challenges in dealing with big data. It supports a wide range of software packages such as Apache Flumes, Apache Oozie, Apache HBase, Apache Sqoop, Apache Spark, Apache Storm, Apache Pig, Apache Hive, Apache Phoenix, Cloudera Impala .

- **Hive** is a data warehouse infrastructure that provides data summarization and ad-hoc querying. It uses HiveQl for data structuring and for writing complicated MapReduce in HDFS .
- **Pig** is a high-level data-flow language and execution framework for parallel computation.
- **HBase** is a distributed database that was designed to store structured data in tables that could have billions of row and millions of columns. It is scalable, distributed, and NoSQL database that is built on top of HDFS.

Hive and Pig are the two integral parts of the Hadoop ecosystem, both of which enable the processing and analyzing of large datasets. There are some critical differences between them both.

Hadoop Eco System Frameworks

Hadoop is a framework that enables processing of large data sets which reside in the form of clusters. Being a framework, Hadoop is made up of several modules that are supported by a large ecosystem of technologies.

- **Introduction**: Hadoop Ecosystem is a platform or a suite which provides various services to solve the big data problems. It includes Apache projects and various commercial tools and solutions.
- **Major Elements**: There are four major elements of Hadoop i.e. HDFS, MapReduce, YARN, and Hadoop Common.

- **Collection of Tools**: The Hadoop Ecosystem is a collection of tools, libraries, and frameworks that help you build applications on top of Apache Hadoop.
- **Parallelism**: Hadoop provides massive parallelism with low latency and high throughput, which makes it well-suited for big data problems.
- **Data Management**: Hadoop is comprised of various tools and frameworks that are dedicated to different sections of data management, like storing, processing, and analyzing.
- **Distributed Processing**: The Apache Hadoop software library is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models.
- **Scalability**: It is designed to scale up from single servers to thousands of machines, each offering local computation and storage.

Applications on Big Data using Pig

Apache Pig is a high-level platform for creating programs that run on Apache Hadoop. The language for this platform is called Pig Latin. Pig can execute its Hadoop jobs in MapReduce, Apache Tez, or Apache Spark.

Some of the applications of Big Data using Pig are:

1. **Data Processing:** Pig is used for data processing tasks such as ETL (Extract, Transform, Load), data preparation, and data analysis.
2. **Log Analysis:** Pig is used for log analysis, which involves processing large volumes of log data to extract useful information.
3. **Ad-hoc Querying:** Pig is used for ad-hoc querying of large datasets. It allows users to write complex data transformations and analysis tasks using a simple scripting language.
4. **Iterative Processing:** Pig is used for iterative processing, which involves processing data in multiple passes to extract more information.
5. **Research:** Pig is used in research for processing large datasets to extract useful information.
6. **Machine Learning:** Pig is used in machine learning for data preparation and feature extraction.
7. **Data Mining:** Pig is used in data mining for data preparation and analysis.
8. **Data Cleansing:** Pig is used for data cleansing, which involves removing or correcting inaccurate, incomplete, or irrelevant data.
9. **Data Integration:** Pig is used for data integration, which involves combining data from multiple sources to provide a unified view.

10. **Data Enrichment:** Pig is used for data enrichment, which involves adding additional information to data to make it more useful.

Applications on Big Data using Hive

Hive is a data warehousing and SQL-like query language for Apache Hadoop, which enables data summarization, querying, and analysis of large datasets stored in Hadoop compatible file systems. Some of the applications of Big Data using Hive are:

1. **Data Processing**: Hive can be used to process structured and semi-structured data in Hadoop. It provides an SQL-like interface to query data stored in various databases and file systems that integrate with Hadoop.
2. **Data Analysis**: Hive can be used for data analysis, including data mining and machine learning. It provides a simple and convenient way to analyze large datasets using familiar SQL syntax.
3. **Data Reporting**: Hive can be used for data reporting, including generating reports and visualizations. It provides a simple and convenient way to generate reports from large datasets using familiar SQL syntax.
4. **Data Integration**: Hive can be used for data integration, including ETL (Extract, Transform, Load) operations. It provides a simple and convenient way to move data between different data sources and Hadoop.
5. **Data Warehousing**: Hive can be used for data warehousing, including storing and managing large datasets. It provides a simple and convenient way to store and manage large datasets in Hadoop.

Overall, Hive is a powerful tool for Big Data applications, providing a simple and convenient way to process, analyze, report, integrate, and store large datasets in Hadoop. It is widely used in various industries, including finance, healthcare, retail, and telecommunications.

Applications on Big Data using HBase

HBase is a distributed, scalable, big data store that is modeled after Google's Bigtable. It is an open-source, non-relational, column-oriented database management system that runs on top of the Hadoop Distributed File System (HDFS). Some of the applications of HBase in big data include:

1. **Real-time data processing:** HBase is designed to handle large amounts of data in real-time. It can be used to store and retrieve data quickly, making it ideal for real-time data processing applications such as log analysis, fraud detection, and recommendation engines.
2. **Data warehousing:** HBase can be used as a data warehouse for storing large amounts of structured and semi-structured data. It can be used to

store data from multiple sources and can be queried using SQL-like syntax, making it easy to analyze and extract insights from the data.

3. **Large-scale data storage:** HBase can be used to store large amounts of data, such as billions of rows and millions of columns. It can be used to store data that is too large to fit in traditional relational databases, making it ideal for big data applications.
4. **Data integration:** HBase can be used to integrate data from multiple sources. It can be used to store data from different systems and can be queried using SQL-like syntax, making it easy to combine and analyze data from different sources.
5. **Time-series data storage:** HBase can be used to store time-series data, such as sensor data, stock market data, and social media data. It can be used to store data with high write and read rates, making it ideal for time-series data storage.

Overall, HBase is a powerful tool for big data applications, providing fast and scalable data storage and processing capabilities. It can be used for a wide range of applications, from real-time data processing to large-scale data storage and analysis.

Pig

- The pig (Sus domesticus), often called swine, hog, or domestic pig when distinguishing from other members of the genus Sus, is an omnivorous, domesticated, even-toed, hoofed mammal.
- It is variously considered a subspecies of Sus scrofa (the wild boar or Eurasian boar) or a distinct species.
- Pigs are stout-bodied, short-legged, omnivorous mammals, with thick skin usually sparsely coated with short bristles.
- Their hooves have two functional and two nonfunctional digits.
- Domestic North American pigs originated from wild stocks still found in European, Asian, and North African forests.
- Any of various mammals of the family Suidae, having short legs, hooves with two weight-bearing toes, bristly hair, and a cartilaginous snout used for digging, including the domesticated hog (Sus scrofa subsp. domestica syn. S. domesticus) and wild species such as the bushpig.

Pig - Introduction to PIG

- Pig is a high-level platform for creating MapReduce programs used with Hadoop.
- It is a data flow language that abstracts the programming from the Java MapReduce idiom into a notation which makes MapReduce programming high level, similar to that of SQL for RDBMS systems.
- Pig Latin is the language used to express data flows in Pig.

- Pig Latin scripts are automatically optimized by the Pig runtime, so the programmer does not have to worry about the execution plan.
- Pig can be used to process structured, semi-structured, and unstructured data.
- Pig can be used interactively or in batch mode.
- Pig can be extended using User Defined Functions (UDFs) written in Java, Python, or other languages.
- Pig is an Apache project, and it is freely available under the Apache license.

Execution Modes of Pig

Pig is a high-level platform for creating MapReduce programs used with Hadoop. It has two execution modes:

1. **Local Mode**: In this mode, Pig runs on a single machine without requiring Hadoop or HDFS. It is used for processing small datasets and for testing and debugging Pig scripts.
2. **MapReduce Mode**: In this mode, Pig runs on a Hadoop cluster and processes data stored in HDFS. It is used for processing large datasets and for production jobs.

In both modes, Pig scripts are translated into a series of MapReduce jobs that are executed on the Hadoop cluster. The choice of execution mode depends on the size of the dataset and the resources available for processing.

Comparison of Pig with Databases

- Pig is a scripting platform that runs on Hadoop clusters, designed to process and analyze large datasets.
- Pig uses a language called Pig Latin, which is similar to SQL.
- This language does not require as much code in order to analyze data.
- Although it is similar to SQL, it does have significant differences.
- Pig makes our life a lot easier, otherwise writing MapReduce is always not easy.
- Pig basically has 2 parts: the Pig Interpreter and the language, PigLatin.
- You write Pig script in PigLatin and using Pig interpreter process them.
- Apache Pig relies on scripts and it requires special knowledge.
- Apache Hive has better access choices and features than that in Apache Pig.
- However, Apache Pig works faster than Apache Hive.

Grunt in Pig

- Grunt is an interactive shell for Apache Pig.
- It is used to execute Pig Latin scripts and Hadoop MapReduce jobs.
- Grunt can be used in local mode or MapReduce mode.

- In local mode, Pig runs on a single machine, while in MapReduce mode, Pig runs on a Hadoop cluster.
- Grunt commands can be used to perform various operations such as loading data, storing data, and defining Pig Latin scripts.
- Grunt also provides several built-in commands for managing files and directories on Hadoop Distributed File System (HDFS).
- Grunt can be launched by running the `pig` command without any arguments.
- Grunt supports command history, tab completion, and command line editing.
- Grunt commands can be executed in batch mode by passing a file containing the commands as an argument to the `pig` command.
- Grunt is a powerful tool for data processing and analysis with Apache Pig.

Pig Latin

Pig Latin is a language game in which words in English are altered. The objective is to conceal the words from others not familiar with the rules. The reference to Latin is a deliberate misnomer, as it is simply a form of jargon, used only for its English connotations as a strange and foreign-sounding language.

Here are the rules for translating English words into Pig Latin: 1. For words that begin with consonant sounds, all letters before the initial vowel are placed at the end of the word sequence. Then, “ay” is added, as in the following examples: - “pig” → “igpay” - “latin” → “atinlay” - “banana” → “ananabay” - “happy” → “appyhay” - “duck” → “uckday” 2. For words that begin with vowel sounds, one just adds “way” or “yay” to the end (or just “ay”). Examples are: - “eat” → “eatway” or “eatay” - “omelet” → “omeletway” or “omeletay” - “are” → “areway” or “areay” - “egg” → “eggway” or “eggay” 3. An alternative convention for words beginning with vowel sounds, one removes the initial vowel(s) along with the first consonant or consonant cluster. This usually only works for words with more than one syllable and offers a more unique variant of the words in question. Examples are: - “every” → “eryevay” - “another” → “otheranay” - “under” → “erunday” - “island” → “andislay” - “elegant” → “egantelay”

User Defined Functions in Pig

- User Defined Functions (UDFs) in Pig allow users to write their own functions to perform operations on data that are not available in Pig’s built-in functions.
- UDFs can be written in Java, Python, Ruby, and other languages.
- UDFs can be used in Pig scripts by registering the JAR file containing the UDF and using the DEFINE keyword to create an alias for the function.
- UDFs can be used in expressions, filters, and other operations in a Pig script.
- UDFs can take one or more input parameters and return a value.

- UDFs can be used to perform complex data transformations, data cleansing, and other operations on data.
- UDFs can be shared and reused by other users.
- UDFs provide flexibility and extensibility to Pig, allowing users to perform custom operations on data that are not available in Pig's built-in functions.

Data Processing Operators in Pig

Pig is a high-level platform for creating MapReduce programs used with Hadoop. It includes a language called Pig Latin, which is used to express data flows. Pig Latin includes several data processing operators that can be used to manipulate and transform data. Here are some of the most commonly used data processing operators in Pig:

1. **LOAD**: This operator is used to load data from the file system into a Pig relation. The data can be in various formats, including text, binary, or sequence files.
2. **STORE**: This operator is used to store the data in a Pig relation to the file system. The data can be stored in various formats, including text, binary, or sequence files.
3. **FILTER**: This operator is used to filter out tuples from a relation based on a specified condition.
4. **FOREACH**: This operator is used to generate a new relation by applying a transformation to each tuple in an input relation.
5. **GROUP**: This operator is used to group the tuples in a relation based on one or more fields.
6. **JOIN**: This operator is used to join two or more relations based on a common field.
7. **ORDER**: This operator is used to sort the tuples in a relation based on one or more fields.
8. **DISTINCT**: This operator is used to remove duplicate tuples from a relation.
9. **LIMIT**: This operator is used to limit the number of tuples in a relation.

These are some of the most commonly used data processing operators in Pig. They can be used to perform a wide range of data manipulation and transformation tasks.

Hive

- Hive is a data warehousing and SQL-like query language for data stored in Hadoop files.
- Hive enables data summarization, querying, and analysis of data.

- Hive queries are written in HiveQL, which is a query language similar to SQL.
- Hive allows you to project structure on largely unstructured data.
- Hive Metastore (HMS) provides a central repository of metadata that can easily be analyzed to make informed, data-driven decisions.
- Hive is built on top of Apache Hadoop and supports storage on S3, adls, gs, etc. through HDFS.

Apache Hive architecture

Apache Hive is a distributed, fault-tolerant data warehouse system that enables analytics at a massive scale . It is an open-source data warehousing tool for performing distributed processing and data analysis . It was developed by Facebook to reduce the work of writing the Java MapReduce program . Apache Hive uses a Hive Query language, which is a declarative language similar to SQL .

The major components of Apache Hive are: - Hive clients - Hive services - Processing framework and Resource Management - Distributed Storage

The key components of the Apache Hive architecture are: - Hive Server 2: accepts incoming requests from users and applications and creates an execution plan and auto generates a YARN job to process SQL queries . - Hive Query Language (HQL) . - External Apache Hive Metastore: provides a central repository of metadata that can easily be analyzed to make informed, data driven decisions, and therefore it is a critical component of many data lake architectures . - Hive Beeline Shell .

Installing Hive

1. Hive is a data warehouse software project built on top of Apache Hadoop for providing data query and analysis.
2. Hive provides a SQL-like interface to stored data in a Hadoop cluster.
3. To install Hive, you must first have a working installation of Hadoop on your system.
4. Download the latest stable release of Hive from the Apache Hive website.
5. Unpack the downloaded tarball and move the resulting directory to a location of your choice.
6. Set the environment variable `HIVE_HOME` to the location of the Hive installation.
7. Add the `$HIVE_HOME/bin` directory to your `PATH` environment variable.
8. Start the Hive shell by running the `hive` command.
9. You can now run HiveQL commands from the Hive shell to interact with your data stored in Hadoop.

Hive Shell

- Hive shell is a command line interface for Apache Hive.
- It is used to interact with the Hive system and execute Hive queries.

- The Hive shell can be accessed by running the `hive` command in the terminal.
- The Hive shell supports various commands, including Data Definition Language (DDL) and Data Manipulation Language (DML) commands.
- DDL commands are used to create, alter, and drop tables, views, and databases.
- DML commands are used to insert, update, and delete data in tables.
- The Hive shell also supports various configuration options, which can be set using the `SET` command.
- The Hive shell provides a convenient way to interact with the Hive system and perform various operations on data stored in Hive tables.

Hive Services

Hive is a data warehousing and SQL-like query language for Hadoop. It provides a mechanism to project structure onto data in Hadoop and to query that data using a SQL-like language called HiveQL. Hive services include:

1. **Hive CLI:** The Hive command line interface (CLI) is a shell where users can enter HiveQL commands and receive results.
2. **HiveServer2:** HiveServer2 is a service that enables clients to execute queries against Hive. It provides a Thrift interface and a JDBC/ODBC server.
3. **Hive Web Interface:** The Hive Web Interface (HWI) is a web-based graphical user interface for Hive.
4. **Hive MetaStore:** The Hive MetaStore stores metadata about the tables, partitions, columns, and storage format of the data in Hive. It can be configured to run in local or remote mode.
5. **Hive Driver:** The Hive Driver manages the lifecycle of a HiveQL statement as it moves through the system. It compiles the statement, optimizes the plan, and coordinates the execution of the plan.
6. **Hive Query Compiler:** The Hive Query Compiler translates HiveQL statements into a directed acyclic graph (DAG) of MapReduce or Tez tasks.
7. **Hive Execution Engine:** The Hive Execution Engine executes the tasks generated by the Hive Query Compiler. It can run on MapReduce or Tez.

Hive metastore

- Hive metastore (HMS) is a service that stores metadata related to Apache Hive and other services, in a backend RDBMS, such as MySQL or PostgreSQL .
- Impala, Spark, Hive, and other services share the metastore .

- The connections to and from HMS include HiveServer, Ranger, and the NameNode that represents HDFS .
- HMS is a central repository of metadata for Hive tables and partitions in a relational database, and provides clients (including Hive, Impala and Spark) access to this information using the metastore service API.
- HMS provides a central repository of metadata that can easily be analyzed to make informed, data-driven decisions, and therefore it is a critical component of many data lake architectures.
- Hive is built on top of Apache Hadoop and supports storage on S3, adls, gs etc though hdfs.
- Hive Metastore was developed as a part of Apache Hive, "a distributed, fault-tolerant data warehouse system that enables analytics at a massive scale".
- Hive achieves this goal by being the storage point for all the meta-information about your data storages.

Comparison of Hive with Traditional Databases

Hive is a data warehouse software system that provides data query and analysis. It gives an interface like SQL to query data stored in various databases and file systems that integrate with Hadoop. Hive helps with querying and managing large datasets real fast.

Here are some key differences between Hive and traditional databases:

- **Schema on Read vs Schema on Write**: Hive applies schema on read time, meaning it does not verify the schema until the data is read. Traditional databases, on the other hand, apply schema on write time, meaning the table schema is enforced when data is inserted.
- **Scalability**: Hive is easily scalable at low cost, while traditional databases are not as scalable and can be costly to scale up.
- **Data Processing**: Hive is based on Hadoop notation, meaning it writes once and reads many times. In traditional databases, data can be read and written multiple times.
- **Record Level Updates**: Record level updates, insertions, and deletions are not possible in Hive, while they are possible in traditional databases.

It is important to note that while Hive resembles a traditional database by supporting an SQL interface, it is not a full database. It can be better called a data warehouse instead of a database. Hive is majorly used to do analysis on a huge amount of data, which traditional databases cannot process using MapReduce. Although for a small number of records, other databases may be faster than Hive. The real power of Hive is unleashed when you have a large number of records, such as 100 million or more. In such cases, Hive performs the query faster than any other database.

HiveQL

HiveQL is a query language used by Apache Hive, a data warehouse system built on top of Hadoop. It is used to analyze large datasets stored in Hadoop's HDFS and compatible file systems such as Amazon S3 filesystem. Some key points about HiveQL are:

1. HiveQL is similar to SQL in terms of syntax and semantics, making it easy for users familiar with SQL to learn and use.
2. HiveQL supports many of the standard SQL operations such as SELECT, WHERE, GROUP BY, HAVING, ORDER BY, and JOIN.
3. HiveQL also supports some advanced features such as partitioning, bucketing, and windowing functions.
4. HiveQL can be used to create, alter, and drop tables, views, and indexes.
5. HiveQL supports user-defined functions (UDFs) and user-defined aggregate functions (UDAFs) to extend its functionality.
6. HiveQL can be used interactively via the Hive command line interface (CLI) or programmatically via the Hive API.
7. HiveQL can be used to process structured, semi-structured, and unstructured data.

HiveQL is a powerful tool for data analysis and can be used to extract insights from large datasets. Its similarity to SQL makes it easy to learn and use, and its advanced features make it a versatile tool for data analysis.

Tables in Hive

Hive is a data warehousing and SQL-like query language for Hadoop. It allows users to create and manage tables in a relational database-like manner. Here are some key points about tables in Hive:

1. **Types of Tables**: There are two types of tables in Hive: managed tables and external tables. Managed tables are created and managed by Hive, while external tables are created and managed by the user.
2. **Creating Tables**: Tables can be created using the `CREATE TABLE` command. The syntax for creating a managed table is `CREATE TABLE table_name (column1 data_type, column2 data_type, ...)`. The syntax for creating an external table is `CREATE EXTERNAL TABLE table_name (column1 data_type, column2 data_type, ...) LOCATION 'hdfs_path'`.
3. **Loading Data**: Data can be loaded into a Hive table using the `LOAD DATA` command. The syntax for loading data into a table is `LOAD DATA [LOCAL] INPATH 'file_path' [OVERWRITE] INTO TABLE table_name`.
4. **Altering Tables**: Tables can be altered using the `ALTER TABLE` command. This command can be used to add or drop columns, change the data type of a column, rename a table, and more.
5. **Dropping Tables**: Tables can be dropped using the `DROP TABLE` command. The syntax for dropping a table is `DROP TABLE [IF EXISTS] table_name`.

Querying Data in Hive

Hive is a data warehousing and SQL-like query language for Hadoop that facilitates easy data summarization, ad-hoc queries, and the analysis of large datasets stored in Hadoop compatible file systems. Here are some points to remember when querying data in Hive:

1. HiveQL is the query language used in Hive, which is similar to SQL.
2. HiveQL supports a wide range of built-in functions, including mathematical, string, and date functions.
3. HiveQL supports subqueries, joins, and group by clauses.
4. HiveQL supports partitioning and bucketing of data to improve query performance.
5. HiveQL supports the creation of views, which can be used to simplify complex queries.
6. HiveQL supports the creation of user-defined functions (UDFs) to extend its functionality.
7. HiveQL supports the use of external tables, which allows querying data stored outside the Hive warehouse.
8. HiveQL supports the use of the `EXPLAIN` keyword to view the query execution plan and optimize queries.

These are some of the key points to remember when querying data in Hive. It is important to have a good understanding of the HiveQL syntax and its capabilities to effectively query and analyze data stored in Hive.

User Defined Functions in Hive

Hive user-defined functions, or UDFs, are custom functions that can be developed in Java, integrated with Hive, and built on top of a Hadoop cluster to allow for efficient and complex computation that would not otherwise be possible with simple SQL. They can be useful and very powerful, and yet online documentation is pretty weak.

In Hive, the users can define their own functions to meet certain client requirements. These are known as UDFs in Hive. User Defined Functions written in Java for specific modules. HIVE UDF (User Defined Functions) allow the user to extend HIVE Query Language. Once the UDF is added in the HIVE script, it works like a normal built-in function. To check which all UDFs are loaded in the current hive session, we use the SHOW command.

Basically, we can use two different interfaces for writing Apache Hive User Defined Functions. Simple API Complex API As long as our function reads and returns primitive types, we can use the simple API (org.apache.hadoop.hive.ql.exec.UDF). In other words, it means basic Hadoop & Hive writable types.

You can configure the cluster to find the JAR using the ADD JAR command on the Hive.

Sorting and Aggregating in Hive

Hive is a data warehousing and SQL-like query language for Hadoop, which allows for easy data summarization, ad-hoc queries, and the analysis of large datasets stored in Hadoop compatible file systems. Hive supports various functions for sorting and aggregating data, which can be used to perform complex data analysis.

- **Sorting**: Hive supports the `ORDER BY` and `SORT BY` clauses for sorting data. The `ORDER BY` clause sorts the data globally, while the `SORT BY` clause sorts the data within each reducer. The `DISTRIBUTE BY` clause can be used in conjunction with the `SORT BY` clause to control the distribution of data to the reducers.
- **Aggregating**: Hive supports various aggregate functions such as `SUM`, `COUNT`, `AVG`, `MIN`, and `MAX`, which can be used to perform calculations on a group of rows. The `GROUP BY` clause can be used to group the rows based on one or more columns, and the aggregate functions can be applied to the grouped data.

These are some of the basic sorting and aggregating functions available in Hive. By using these functions, users can perform complex data analysis and extract meaningful insights from large datasets.

Map Reduce scripts in Hive

MapReduce is a programming model for processing large data sets in parallel across a distributed cluster of processors or stand-alone computers. It was developed by Google and is now an integral part of the Apache Hadoop project.

Hive is a data warehousing and SQL-like query language for Hadoop that facilitates easy data summarization, ad-hoc queries, and the analysis of large datasets stored in Hadoop compatible file systems. Hive provides a mechanism to project structure onto this data and query the data using a SQL-like language called HiveQL.

Hive can use custom MapReduce scripts to process data. These scripts can be written in any language that can read from standard input and write to standard output. The scripts are specified in the TRANSFORM clause of the SELECT, GROUP BY, or MAPREDUCE statements.

Here are the steps to use MapReduce scripts in Hive:

1. Write the MapReduce script in the desired language and save it to a file.
2. Use the ADD FILE command to add the script file to the distributed cache so that it can be accessed by all nodes in the cluster.
3. Use the TRANSFORM clause in the SELECT, GROUP BY, or MAPREDUCE statements to specify the script and its input and output formats.
4. Run the query to execute the MapReduce script on the data.

Example:

```
ADD FILE /path/to/mapper.py;
ADD FILE /path/to/reducer.py;

SELECT TRANSFORM (columns)
USING 'python mapper.py'
AS (output_columns)
FROM input_table
CLUSTER BY columns
```

This example adds the mapper.py and reducer.py scripts to the distributed cache, then uses the TRANSFORM clause to specify the mapper script and its input and output formats. The CLUSTER BY clause is used to ensure that the data is partitioned correctly for the reduce phase.

In summary, Hive provides a powerful and flexible way to use custom MapReduce scripts to process data. By following the steps outlined above, you can easily integrate your own scripts into your Hive queries.

Joins and Subqueries in Hive

Hive is a data warehousing and SQL-like query language for Hadoop that facilitates easy data summarization, ad-hoc queries, and the analysis of large datasets stored in Hadoop compatible file systems. Hive supports various types of joins and subqueries, which are used to combine and analyze data from multiple tables.

1. **Joins in Hive:** Joins are used to combine rows from two or more tables based on a related column between them. Hive supports several types of joins, including inner join, left outer join, right outer join, full outer join, and cross join.
    - **Inner Join:** Returns only the rows from both tables that satisfy the join condition.
    - **Left Outer Join:** Returns all the rows from the left table and the matching rows from the right table. If there is no match, the result will contain null for all columns of the right table.
    - **Right Outer Join:** Returns all the rows from the right table and the matching rows from the left table. If there is no match, the result will contain null for all columns of the left table.
    - **Full Outer Join:** Returns all the rows from both tables. If there is no match, the result will contain null for all columns of the table without a match.
    - **Cross Join:** Returns the Cartesian product of the two tables, i.e., each row of the first table is combined with each row of the second table.
2. **Subqueries in Hive:** A subquery is a query that is nested inside another

query. Hive supports subqueries in the WHERE and HAVING clauses, as well as in the FROM clause.

- **Subqueries in the WHERE clause:** A subquery in the WHERE clause can be used to filter the rows returned by the main query based on the results of the subquery.
- **Subqueries in the HAVING clause:** A subquery in the HAVING clause can be used to filter the groups returned by the main query based on the results of the subquery.
- **Subqueries in the FROM clause:** A subquery in the FROM clause can be used to create a derived table that can be used in the main query.

These are some of the basic concepts of joins and subqueries in Hive. They can be used to perform complex data analysis and manipulation on large datasets stored in Hadoop compatible file systems.

HBase

- HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS).
- It provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases.
- HBase is well suited for real-time data processing or random read/write access to large volumes of data.
- It is ideal for high-scale real-time applications, such as a social media app or a streaming application.
- Thanks to the lack of a fixed database schema in a non-relational database like HBase, developers can add new data without conforming to a schema model.
- Apache HBase is an open-source, distributed, versioned, non-relational database modeled after Google's Bigtable.
- Just as Bigtable leverages the distributed data storage provided by the Google File System, Apache HBase provides Bigtable-like capabilities on top of Hadoop and HDFS.
- HBase is an open-source non-relational distributed database modeled after Google's Bigtable and written in Java.
- It is developed as part of Apache Software Foundation's Apache Hadoop project and runs on top of HDFS (Hadoop Distributed File System) or Alluxio, providing Bigtable-like capabilities for Hadoop.

HBase Concepts

HBase is a distributed, column-oriented database that is built on top of the Hadoop Distributed File System (HDFS). It is an open-source project that is part of the Apache Hadoop ecosystem. Here are some key concepts to understand when working with HBase:

1. **Column Families**: HBase organizes data into column families, which are groups of related columns. Each column family is stored separately on disk, so it is important to group columns that are frequently accessed together into the same column family.
2. **Rows**: HBase stores data in rows, with each row identified by a unique row key. Rows are sorted lexicographically by their row key, which allows for fast retrieval of data for a specific row or range of rows.
3. **Cells**: A cell in HBase is the intersection of a row and a column. Each cell can store multiple versions of a value, with each version identified by a timestamp.
4. **Regions**: HBase automatically splits large tables into smaller units called regions, which are distributed across the cluster. Each region is responsible for a specific range of row keys and is served by a single region server.
5. **Region Servers**: Region servers are responsible for managing the regions assigned to them. They handle read and write requests from clients and communicate with the Hadoop Distributed File System (HDFS) to store and retrieve data.
6. **HMaster**: The HMaster is responsible for coordinating the cluster and managing the assignment of regions to region servers. It also handles administrative tasks such as creating and deleting tables.
7. **ZooKeeper**: HBase uses Apache ZooKeeper, a distributed coordination service, to manage the cluster state and ensure that all nodes are aware of the current configuration.

These are some of the key concepts to understand when working with HBase. By understanding these concepts, you can better design and implement your HBase applications.

HBase Clients

HBase is a distributed, scalable, big data store that runs on top of the Hadoop Distributed File System (HDFS). It is an open-source, non-relational, column-oriented database management system that is modeled after Google's Bigtable. HBase clients are used to interact with the HBase database.

Here are some key points about HBase clients:

1. HBase clients are used to perform operations such as creating, reading, updating, and deleting data in HBase tables.
2. HBase provides a Java API for clients to interact with the database. This API is used by Java applications to perform operations on HBase tables.
3. HBase also provides a REST API and a Thrift API for non-Java clients to interact with the database. These APIs can be used by applications written in languages such as Python, Ruby, and PHP to perform operations

on HBase tables.
4. HBase clients can also use the HBase shell, which is a command-line interface, to interact with the database. The HBase shell provides a set of commands that can be used to perform operations on HBase tables.
5. HBase clients can also use Apache Phoenix, which is an SQL skin for HBase, to interact with the database. Apache Phoenix provides a JDBC driver that can be used by applications to perform SQL queries on HBase tables.

In summary, HBase clients are used to interact with the HBase database and perform operations on HBase tables. HBase provides several APIs and interfaces for clients to interact with the database, including a Java API, a REST API, a Thrift API, the HBase shell, and Apache Phoenix. These APIs and interfaces can be used by applications written in various languages to interact with HBase.

HBase example

HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS). It provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases. It is well suited for real-time data processing or random read/write access to large volumes of data.

Here are some examples of how HBase is used in different industries:

1. In the healthcare sector, HBase is used for storing genome sequences and disease history of people or a particular area.
2. In the field of e-commerce, HBase is used for storing logs about customer search history and it also performs analytics and target advertisement for better business insights.
3. In sports, HBase is used to store match details and the history of each match.

An example of HBase in action is when storing diagnostic logs from servers in an environment. Each row might be a log record, and a typical column could be the timestamp of when the log record was written, or the server name where the record originated.

Another example is creating a table in HBase with the specified name and column family. For instance, to create a table named 'education' with a column family 'guru99', the HBase shell command would be: `create 'education','guru99'`.

HBase vs RDBMS

HBase and RDBMS are both types of database management systems, but they differ in several ways:

1. **Data Model**: RDBMS uses a relational data model, where data is stored in tables with predefined columns and rows. HBase, on the other hand, uses a column-family data model, where data is stored in column families, which contain columns and rows. HBase is often referred to as a NoSQL database because of its non-relational data model.
2. **Scaling**: HBase is better suited for big data applications that require horizontal scaling and high-speed processing.
3. **Consistency**: RDBMS is more suitable for traditional, transactional applications that require strong consistency.
4. **Speed**: HBase provides random access and strong consistency for large amounts of unstructured and semi-structured data in a schemaless database organized by column families.
5. **ACID compliance**: RDBMS mostly guarantees transaction integrity, whereas in HBase, there is no transaction guaranty.
6. **JOINs**: HBase supports JOINs, whereas RDBMS does not support JOINs.
7. **Referential integrity**: RDBMS has referential integrity, whereas HBase does not have referential integrity.

In summary, RDBMS and HBase differ in their data model, scaling, consistency, speed, and ACID compliance. RDBMS is more suitable for traditional, transactional applications that require strong consistency, whereas HBase is better suited for big data applications that require horizontal scaling and high-speed processing.

Advanced Usage of HBase

HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS). It provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases. It is well suited for real-time data processing or random read/write access to large volumes of data.

HBase has two fundamental key structures: the row key and the column key. Both can be used to convey meaning, by either the data they store, or by exploiting their sorting order. These keys can be used to solve commonly found problems when designing storage solutions.

Some advanced use cases of HBase include: 1. In the healthcare sector, HBase is used for storing genome sequences and disease history of people or a particular area. 2. In the field of e-commerce, HBase is used for storing logs about customer search history and it also performs analytics and target advertisement for better business insights. 3. In sports, HBase is used to store match details and the history of each match.

HBase is mainly used for random, real-time read/write access to Big Data. It can be used when there is a need to store huge volumes of data and high scalability is required. However, it can only be used if the user can live without all the extra features of traditional database systems like typed columns, transactions, advanced query languages, secondary indexes, etc.

Advanced Indexing in HBase

- HBase is a column-oriented NoSQL database management system that runs on top of the Hadoop Distributed File System (HDFS). It is modeled after Google's Big Table and written in Java.
- In HBase, there are no indexes. The rowkey, column family, column qualifier are all stored in sort order based on the java comparable method for byte arrays.
- Access to records in any way other than through the primary row key requires scanning over potentially all the rows in the table to test them against your filter.
- Secondary indexing is a way to improve the performance of queries that do not use the primary row key. HBASE-9203 is a Jira entry that exists specifically to address the ideas behind secondary indexing.
- An index will surely work faster than scanning 50M rows every time. If you use an HBase version that already has coprocessors you can follow Xodarap advice. If you are using older versions of HBase you need to set up an additional table to act as the index and update manually.
- Lily HBase Indexer service embeds the NG-Data Indexer to provide a Near-Real-Time (NRT) resilient automated configuration-driven mechanism to trigger Morphline (Kite SDK) parsers over an HBase table.

Zookeeper

Zookeeper is a centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services. All of these kinds of services are used in some form or another by distributed applications. It is essentially a service for distributed systems offering a hierarchical key-value store, which is used to provide a distributed configuration service, synchronization service, and naming registry for large distributed systems. ZooKeeper was a sub-project of Hadoop but is now a top-level Apache project in its own right.

- Zookeeper is a centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services.
- It is used by distributed applications.
- It offers a hierarchical key-value store.
- It provides a distributed configuration service, synchronization service, and naming registry for large distributed systems.

- ZooKeeper was a sub-project of Hadoop but is now a top-level Apache project in its own right.

Zookeeper concepts

Apache ZooKeeper is an open-source server that enables highly reliable distributed coordination. It is a centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services. Here are some key concepts of ZooKeeper:

1. **Znodes**: ZooKeeper stores data in a hierarchical namespace, much like a file system. The nodes in this namespace are called znodes. Each znode can store a small amount of data and has an associated stat structure that includes version numbers for data changes, ACL changes, and timestamps.
2. **Watches**: Clients can set watches on znodes. A watch is a one-time trigger that notifies the client when the znode changes.
3. **Ephemeral nodes**: Ephemeral nodes are znodes that exist as long as the session that created them is active. When the session ends, the znode is deleted.
4. **Sequential nodes**: Sequential nodes are znodes that are created with a unique monotonically increasing sequence number appended to the path name.
5. **Consistency**: ZooKeeper guarantees that once a write is complete, all subsequent reads will see that write. It also guarantees that updates from a client will be applied in the order that they were sent.
6. **Atomicity**: All updates are atomic. Either all the changes are applied, or none of them are.
7. **Reliability**: ZooKeeper is designed to be highly reliable. It replicates its data over a set of servers, and as long as a majority of the servers are available, ZooKeeper will be available.

How Zookeeper helps in monitoring a cluster

Zookeeper is a tool that helps in maintaining configuration information, naming, and group services for distributed applications. It implements different protocols on the cluster so that the application should not implement on their own. It provides a single coherent view of multiple machines.

Some of the ways Zookeeper helps in monitoring a cluster are:

1. **Maintaining Configuration Information**: Zookeeper helps to maintain configuration information for distributed applications.
2. **Group Services**: Zookeeper helps to maintain group services for distributed applications.

3. **Implementing Protocols**: Zookeeper implements different protocols on the cluster so that the application should not implement on their own.
4. **Single Coherent View**: Zookeeper provides a single coherent view of multiple machines.
5. **Node Count Consistency**: Applications Manager's ZooKeeper monitoring helps make sure the total node count inside the ZooKeeper tree is consistent.
6. **Thread and JVM usage**: Zookeeper can help to analyze a JVM Thread Dump and pinpoint the root cause of issues.
7. **Prometheus MetricsProvider**: Running a Prometheus monitoring service is the easiest way to ingest and record ZooKeeper's metrics.

How to build applications with Zookeeper

Zookeeper is a distributed system coordinator that is used to provide highly-available services and to make distributed programming easier. It is used by Apache HBase, HDFS, and other Apache Hadoop projects .

1. **Starting ZooKeeper and Application Builder**: After installing Application Builder in admin mode, execute the `/etc/init.d/zookeeper-service-default` on the server where ZooKeeper is installed . In non-admin mode, start the ZooKeeper server before starting Application Builder. To start the ZooKeeper server on a Linux system, use the `Zookeeper/zookeeper/bin/zkServer.sh restart` command from your Watson Explorer installation directory .
2. **Running ZooKeeper on Kubernetes**: Open a terminal, and use the `kubectl apply` command to create the manifest. `kubectl apply -f https://k8s.io/examples/application/zookeeper/zookeeper.yaml`. This creates the `zk-hs` Headless Service, the `zk-cs` Service, the `zk-pdb` PodDisruptionBudget, and the `zk` StatefulSet .
3. **Setting up a ZooKeeper server in standalone mode**: The server is contained in a single JAR file, so installation consists of creating a configuration. Once you've downloaded a stable ZooKeeper release, unpack it and `cd` to the root. To start ZooKeeper you need a configuration file .

IBM Big Data strategy

IBM, a US-based computer hardware and software manufacturer, has implemented a Big Data strategy. The company offers solutions to store, manage, and analyze the huge amounts of data generated daily and equips large and small companies to make informed business decisions .

- IBM's Big Data strategy is part of its corporate initiative called "Smarter Planet", which seeks to highlight how government and business leaders

are capturing the potential of smarter systems to achieve economic and sustainable growth and societal progress.

- IBM's data strategy framework consists of six steps: understanding business objectives, assessing the current state, mapping out the data strategy framework, defining the data target, identifying the data sources, and implementing the data strategy.
- IBM also offers Big Data analytics solutions, such as an enterprise-grade, secure, governed, open source-based data lake, and partnerships with companies like Cloudera to connect the data lifecycle and accelerate the journey to hybrid cloud and AI.
- IBM recommends giving data assets and accelerators top priority, developing a process and culture around data that enables true standardization, re-use, portability, speed to action, and risk reduction across the end-to-end data lifecycle.

IBM Big Data strategy

IBM, a US-based computer hardware and software manufacturer, had implemented a Big Data strategy. The company offered solutions to store, manage, and analyze the huge amounts of data generated daily and equipped large and small companies to make informed business decisions .

- IBM's Big Data strategy is part of its Smarter Planet corporate initiative, which sought to highlight how government and business leaders were capturing the potential of smarter systems to achieve economic and sustainable growth and societal progress.
- IBM's data strategy framework consists of six steps: understanding business objectives, assessing the current state, mapping out the data strategy framework, defining the data target, identifying the data sources, and implementing the data strategy.
- IBM also provides Big Data analytics solutions, such as an enterprise-grade, secure, governed, open source-based data lake, and partnerships with companies like Cloudera to connect the data lifecycle and accelerate the journey to hybrid cloud and AI.
- IBM recommends giving data assets and accelerators top priority, developing a process and culture around data that enables true standardization, re-use, portability, speed to action, and risk reduction across the end-to-end data lifecycle.

Introduction to Infosphere

1. Infosphere is a term used to describe the information environment that surrounds us.

2. It includes all the information that is available to us, including data, text, images, and sounds.
3. The infosphere is constantly evolving and expanding as new information is created and shared.
4. It is accessible through various means, including the internet, television, radio, and print media.
5. The infosphere plays a crucial role in our daily lives, shaping our perceptions, decisions, and actions.
6. It is important to be able to navigate and critically evaluate the information in the infosphere in order to make informed decisions.
7. The study of the infosphere and its impact on society is an interdisciplinary field, drawing on disciplines such as information science, communication studies, and sociology.

Introduction to BigInsights

BigInsights is an IBM distribution of Apache Hadoop, a software framework for distributed processing of large data sets across clusters of computers. It is designed to handle data from a variety of sources, including structured, semi-structured, and unstructured data.

Some key features of BigInsights include: - Integration with other IBM products, such as IBM Cognos and IBM SPSS. - Support for multiple data sources, including Hadoop Distributed File System (HDFS), HBase, and relational databases. - Tools for data analysis, including BigSheets, a spreadsheet-like interface for exploring and analyzing data. - Support for multiple programming languages, including Java, Python, and R. - Security features, including Kerberos authentication and data encryption.

BigInsights is used in a variety of industries, including finance, healthcare, and retail, to analyze large amounts of data and gain insights into customer behavior, market trends, and other business-critical information. It is a powerful tool for data analysis and can help organizations make more informed decisions.

Introduction to Big Sheets

Big Sheets is a cloud-based tool that allows users to analyze and visualize large amounts of data. Some of the key features of Big Sheets include:

1. **Scalability:** Big Sheets can handle large amounts of data, making it ideal for big data analysis.
2. **Ease of use:** Big Sheets has a user-friendly interface that makes it easy to use, even for those without a technical background.
3. **Collaboration:** Big Sheets allows multiple users to work on the same data set, making it a great tool for collaborative data analysis.
4. **Visualization:** Big Sheets has a variety of visualization options, allowing users to create charts and graphs to better understand their data.

Overall, Big Sheets is a powerful tool for data analysis and visualization, and is well-suited for handling large data sets.

Introduction to Big SQL

Big SQL is a high-performance SQL engine that enables you to query and analyze data stored in Hadoop. It is a component of IBM's BigInsights platform and provides a familiar SQL interface for data analysts and developers to work with Hadoop data.

Some key features of Big SQL include: - Support for a wide range of data sources, including Hadoop Distributed File System (HDFS), HBase, and other relational databases. - Integration with other BigInsights components, such as BigSheets and Big R, for data analysis and visualization. - Support for ANSI SQL and compatibility with existing SQL-based tools and applications. - High performance through the use of advanced query optimization techniques and parallel processing.

Big SQL provides a powerful and flexible way to work with big data stored in Hadoop, allowing you to leverage your existing SQL skills and tools to analyze and gain insights from your data.